

THE COMPLETE PRACTITIONER'S COURSE

Big Data Analytics & Machine Learning

From SQL, Excel and SPSS to Spark, Deep Learning
and Production AI

Tatenda Makuvaza

"The Big Data Analyst"

First Edition · 2026

Table of Contents

Front Matter	3
The Extended Curriculum: Special Topics and the Craft of Teaching	7
Chapter 49: Building Data Products — Dashboards, Apps, and APIs	8
Chapter 50: Teaching Analytics — A Course Design and Instructor's Guide	13
Analytics Across Industries: Domain Playbooks	20
Chapter 51: Retail and E-Commerce — The Ndineka Fashion House Playbook	21
Chapter 52: Marketing and Customer Analytics — The Zuva Mobile Playbook	26
Chapter 53: Financial Services — The ZuvaPay Playbook	30
Chapter 54: Healthcare Analytics — The Batanai Health Trust Playbook	34
Chapter 55: Agriculture and Food Security — The Chikafu Foods and NGA Playbook	38
Chapter 56: Energy Analytics — The Sable Energy Playbook	42
Chapter 57: Government and Public Sector Analytics — The Harare Metro Playbook	46
Chapter 58: Manufacturing Analytics — The Sabvura Industries Playbook	50
Chapter 59: Transport and Logistics — The Kunaka Logistics Playbook	55
Chapter 60: Media and Sport Analytics — The Nhaka FC and Kumba Playbook	59
Reference Section	63
Appendix E: Datasets and Resources	64
Appendix I: Instructor Slide Kit	68
Appendix J: Extended Case Portfolio	74
About this Edition	82

Front Matter

About This Book

Big Data Analytics and Machine Learning — The Complete Practitioner's Course: From SQL, Excel and SPSS to Spark, Deep Learning and Production AI

Tatenda Makuvaza — "The Big Data Analyst"

First Edition, 2026

How This Book Came to Be

I completed a six-module data analytics course — SQL, Excel, Python, Power BI, Stata and SPSS — and then faced the question every graduate faces: *what now?* The tools were in my hands, but the path from "I know some SQL and I can build a dashboard" to "I can run an analytics practice, teach a class, and ship a machine-learning system people trust" was nowhere written down. This book is the path, written down — twelve parts, eighty chapters, and a set of appendices designed to be taught from as readily as they are read.

Every chapter is built on the same bridge: **From Your Toolkit** callouts connect each new idea back to the six foundation modules, because you never learn analytics by abandoning what you know — you learn it by extending it. And every concept lives in one running world: **Soulfya's Restaurant Group**, a fictional three-outlet restaurant chain in Harare whose data carries the whole book, from its first spreadsheet to its production ML systems.

Who This Book Is For

- **Graduates of foundation courses** who know the six tools and need the disciplines that turn tools into practice.
- **Working analysts** who want to move from reports to models, and from models to systems.
- **Instructors** building a course: the book is sequenced as a full syllabus (below), with a slide kit, an assessment bank, worked solutions, and a complete instructor's guide in Chapter 50.
- **Self-taught practitioners** who want their knowledge to have the same foundations a degree would have given them.

The Arc of the Book

Part	Question it answers	What it holds
I — Foundations	What is analytics, and when is data actually big?	Questions, maturity, the Vs, data types, your six tools
II — Data Engineering	Where does trustworthy data come from?	Grain, joins, pipelines, warehouses, cleaning, quality

Part	Question it answers	What it holds
III — Math and Statistics	How do I argue with numbers honestly?	Descriptives, probability, sampling, testing, regression
IV — Visualization and BI	How do I make numbers change minds?	Chart craft, metric trees, dashboards, storytelling
V — Machine Learning	How do I make data predict?	Framing, leakage, baselines, models, evaluation, ethics
VI — Advanced ML	What lies beyond the first model?	Clustering, recommenders, NLP, deep learning, LLMs, explanation
VII — Big Data and Cloud	What changes at scale?	The decision test, DuckDB, Spark, streaming, cloud economics
VIII — Insight to Impact	How do we run this responsibly, forever?	Governance, MLOps, monitoring, ethics, careers
IX — Extended Curriculum	How do we go further — and teach it?	Anomaly detection, optimization, graphs, data products, teaching
X — Domain Playbooks	How is analytics actually used, industry by industry?	Ten sector playbooks with metrics, methods and running cases
XI — The Frontier	What lies beyond the working core?	Causality, space, time, language, simulation, privacy, staying current
XII — The Practitioner's Path	How is analytics actually delivered?	Projects, engagements, interviews, storytelling, documentation, audits, constraint, onboarding and the business of the craft

The book has twelve parts and eighty chapters. Parts build on each other, but each part is designed to be coherent on its own, so a course can enter at Part III (statistics) or Part V (machine learning) if the foundations are already in place. Part IX (Chapters 46–50) extends the curriculum with special topics — anomaly detection, optimization, graph analytics, and data products — and closes with a complete instructor's guide. Part X (Chapters 51–60) is a set of ten industry playbooks — retail, marketing, financial services, healthcare, agriculture, energy, transport, manufacturing, government, and media and sport. Part XI (Chapters 61–70) is the frontier: causal inference, geospatial analytics, survival modelling, hierarchical and probabilistic forecasting, language analytics for low-resource languages, simulation and synthetic data, experimentation at scale, privacy engineering, analytics engineering, and staying current. Part XII (Chapters 71–80) is the practitioner's path — the delivery craft.

How to Read This Book

Three habits, sustained across eighty chapters, are worth more than any tool in it:

1. **Ask the decision question before the data question.** Every chapter's first move is "what will someone do differently?"
2. **Name a baseline before praising a result.** Seasonal-naive, majority class, last year: if you cannot beat the cheapest alternative, use the alternative.
3. **Write for a reader who is not you.** Memos, model cards, logs: the artefact is the analysis.

Read with a lab open. Every chapter ends with a Practice Lab, and the labs are load-bearing: the prose is scaffolding for the labs, not the other way around.

A Fourteen-Week Course Plan

Week	Chapters	Theme	Lab focus
1	1-3	Questions, maturity, the Vs	Data inventory
2	4-5	Data types; your six tools bridged	JSON and sources
3	6-7	Joins and SQL craft	Fan-out demo
4	8-9	Grain, pipelines	Star schema
5	10-11	pandas-to-SQL; cleaning and quality	Validation gates
6	12-13	Descriptives; probability by counting	Bayes table
7	14-15	Testing; sampling and resampling	Simulation lab
8	16-17	Regression; visualization craft	Chart audit
9	18-20	Metric trees, dashboards, storytelling	One-page dashboard
10	21-23	Framing, leakage, linear models	Framing artefact
11	24-25	Evaluation; regularized regression	Baselines and metrics
12	26-28	Forecasting; trees; logistic models	MASE league table
13	29 + 30-32	Ensembles; clustering, recommenders	League table; segments
14	33-36 (skim 37-45)	Uplift, forecasting depth, NLP, deep learning, LLMs	Capstone presentation
Extension term	46-80 + Apps I-J	Special topics, domain playbooks, frontier methods and the delivery craft; case-method seminars from Appendix J	Extended capstone or honours seminar

Chapters 37-80 make an excellent second-semester or honours extension: Spark, streaming, cloud, governance, MLOps, case studies and careers; Part IX's special topics with the full instructor's guide of Chapter 50; Part X's ten domain playbooks, best taught as case-method seminars with Appendix I's slide kits and Appendix J's five end-to-end cases; Part XI's frontier chapters for honours students, one chapter and one paper per week; and Part XII's delivery craft, best taught through its role-plays.

The Appendices

Ten appendices back the course: **A** a Python crash course; **B** a SQL reference card; **C** an algorithm quick-reference; **D** a glossary; **E** datasets and resources; **F** the Soulfya's practice-dataset generator (a script that fabricates the entire running case study's data with planted patterns); **G** worked solutions to selected practice labs; **H** the assessment bank (midterm, final and viva questions with marking guidance); **I** the instructor slide kit; and **J** the extended case portfolio.

A Note on the Running Case

Soulfya's Restaurant Group is fictional, and so is everyone in it — but nothing about its data is arbitrary. Its outlets (Avondale, Borrowdale, Bulawayo), its Soul Circle loyalty programme, its delivery channel and its reviews carry every pattern this book teaches: weekly seasonality, payday spikes, promo lifts, outlet differences, a genuine churn signal. By Part V you will model its customers; by Part VII you will scale its pipelines; by Part XII you will run its projects. If the chain feels real by Chapter 10, the book is working.

— *Tatenda Makuva*, "The Big Data Analyst", Harare, 2026

PART IX

The Extended Curriculum: Special Topics and the Craft of Teaching

Chapters 46 – 50

Chapter 49: Building Data Products — Dashboards, Apps, and APIs

Part IX — The Extended Curriculum: Special Topics and the Craft of Teaching

"The last mile of analytics is a product, not a PDF."

In this chapter you will learn

- The product ladder: report → dashboard → app → API → automated decision service — and when to climb each rung.
- A working Streamlit app in one file: the churn explorer.
- A production-shape scoring API with FastAPI.
- The product crafts: versioning, logging, documentation, authentication.
- Deployment realities: containers, CI, and what to run where.
- When *not* to build a product — the discipline of the last rung.

49.1 The Product Ladder

Analysis creates understanding; a product *delivers* it, repeatedly, to someone who was not in the room. Each rung of the ladder adds interactivity, engineering, and maintenance — climb only as far as the decision demands:

Ru ng	Form	Interaction	Maintenance	Soulfya's example
1	Report / memo	None	None	Monthly performance write-up
2	Dashboard (Chapter 18–19)	Filter and drill	Refresh, model upkeep	"Are we healthy?" page
3	Notebook-as-app	Parameter play	Low	Scenario planner for the founder
4	Web app	Full interaction	Real	Churn campaign workbench
5	API	Machine-to-machine	Real + latency SLAs	Live delivery-ETA scoring
6	Automated decision service	Acts on its own	Everything in Part VIII's MLOps	Dynamic prep recommendations

The professional error is skipping rungs: teams build a web app (rung 4) for an audience that needed a well-written memo (rung 1), or an API (rung 5) for a question asked quarterly. The other professional error is *camping* at rung 1 forever while a weekly, repeated, human-heavy ritual begs for rung 2.

49.2 Rung 3-4 in One File: Streamlit

Streamlit turns a Python script into a web app: the script reruns top-to-bottom on every interaction, widgets become function calls, and the whole framework fits in a morning. The churn campaign workbench:

```
import streamlit as st
import pandas as pd
import joblib

st.set_page_config(page_title="Churn Explorer", layout="wide")
st.title("Soul Circle Churn Explorer")

@st.cache_data
def load_scores():
    scores = pd.read_csv("member_scores.csv", parse_dates=["score_date"])
    scores["segment"] = scores["segment"].fillna("unknown")
    return scores

@st.cache_resource
def load_model():
    return joblib.load("churn_pipeline.joblib")

scores = load_scores()
model = load_model()

with st.sidebar:
    outlets = st.multiselect("Outlets", sorted(scores["outlet_id"].unique()))
    min_risk = st.slider("Minimum risk score", 0.0, 1.0, 0.5)
    top_n = st.number_input("Campaign size", 50, 2000, 200)
    run = st.button("Build campaign")

view = scores[scores["risk"] >= min_risk]
if outlets:
    view = view[view["outlet_id"].isin(outlets)]

col1, col2, col3 = st.columns(3)
col1.metric("Members above threshold", f"{len(view):,}")
col2.metric("Churners in view (last 60d)", f"{view['churned'].sum():,}")
col3.metric("Est. value of campaign", f"${view.nlargest(top_n,
    'risk')['expected_value'].sum():.0f}")

st.scatter_chart(view, x="days_since_visit", y="risk", color="segment", height=380)

if run:
    campaign = view.nlargest(top_n, "risk")
    campaign.to_csv("this_weeks_campaign.csv", index=False)
    st.success(f"Campaign built: {len(campaign)} members, "
        f"{campaign['member_id'].nunique()} unique, CSV ready for download.")
    st.data_editor(campaign[["member_id", "risk", "days_since_visit", "segment"]])
```

Fifteen minutes of work produces what a month of email round-trips never did: the retention team explores risk thresholds themselves, sees the expected value move, and pulls the campaign list when they are satisfied. The `@st.cache_data` / `@st.cache_resource` decorators are the entire performance strategy — data loads once, the model loads once.

The honest limits: Streamlit apps are internal tools. Authentication is basic, multi-user state is awkward, and anything customer-facing eventually wants a real frontend — by which point the app has already paid for itself as the prototype that proved the need.

49.3 Rung 5: A Scoring API with FastAPI

When another system needs your model — the delivery app's backend, the POS, a partner — you serve it as an API. FastAPI is the modern standard: type hints become validation, and documentation is generated automatically.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
import joblib, pandas as pd

app = FastAPI(title="Soulfyas Scoring API", version="1.2.0")
model = joblib.load("churn_pipeline.joblib")

class MemberFeatures(BaseModel):
    days_since_visit: float = Field(..., ge=0, le=730)
    visits_90d: int = Field(..., ge=0)
    avg_basket: float = Field(..., ge=0)
    delivery_share: float = Field(..., ge=0, le=1)
    complaint_filed: int = Field(..., ge=0, le=1)

class ChurnScore(BaseModel):
    risk: float
    model_version: str

@app.post("/v1/churn", response_model=ChurnScore)
def score(m: MemberFeatures):
    X = pd.DataFrame([m.model_dump()])
    risk = float(model.predict_proba(X)[0, 1])
    return ChurnScore(risk=risk, model_version="1.2.0")

# run: uvicorn scoring_api:app --host 0.0.0.0 --port 8000
```

Read what the fifteen lines include: input *validation* (impossible values rejected with a 422 before they reach the model), a *versioned route* (`/v1/` — so v2 can ship beside v1), and the model version in every response — Chapter 43's traceability requirements, implemented as a side effect of good design. Swagger documentation appears at `/docs` for free. In production, add authentication (an API key header at minimum), structured logging of inputs and scores (the drift dataset of Chapter 43), and latency monitoring.

49.4 Product Craft: The Four Habits

1. **Version everything together** — code (git), model artefact (registry), API route (`/v1/`), and the version echoed in responses. "Which model produced this score?" must be answerable from the score itself.
2. **Log like you will be debugged** — every request (timestamp, features, score, latency, caller), because your monitoring data *is* your log, and Chapter 43's drift detection will read exactly this table.
3. **Document the contract** — the API's generated docs plus one human page: what the score means, its calibration, its known weaknesses (Chapter 22's model card, linked from the product).
4. **Degrade gracefully** — a model failure should not take down the caller: fall back to a default score, a cached result, or an explicit "unavailable" that the caller is designed to handle. Products fail; *systems* degrade.

49.5 Deployment, Minimally

The container that runs the same everywhere (Chapter 43's one-sentence Docker):

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY scoring_api.py churn_pipeline.joblib .
CMD ["uvicorn", "scoring_api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Where it runs is a Chapter 41 decision: a small VM, a container service, or a platform's managed endpoint — the artefact does not change. The CI habit from Chapter 43 applies verbatim: every push runs the model smoke test (fit on a fixed toy sample, score a fixed row, assert the score within tolerance) before anything deploys.

49.6 The Discipline of the Last Rung

The rung you choose is a maintenance contract. A memo costs nothing after Thursday; a dashboard costs a refresh schedule and a model; an API costs latency, uptime, and a rotation of on-call analysts. Before climbing, ask Chapter 2's question in product form: *how many people, how often, making which decision?* Soulfya's churn workbench served three people weekly — a Streamlit app, aggressively unglamorous, was the correct ceiling. The delivery-ETA scorer served thousands of app sessions daily — a containerised API earned itself. Both answers are right; the sin is not knowing which you are building.

Teaching Tip — Ship week: make the course capstone literally a shipped artefact — a live URL (Streamlit sharing or any free host) or a running container — graded with a rubric of product habits: does it state its model version? Does it degrade gracefully? Would a stranger understand the front page in thirty seconds? Students who have *deployed something* interview differently: they have war stories about CORS, caching, and cold starts, and war stories are what "experience" means.

Key Takeaways

- Products are rungs, not a summit: report → dashboard → app → API → decision service; climb to match the decision's frequency and audience, no further.
- Streamlit turns analysis into an interactive workbench in an afternoon — the highest value-per-line tool in this book for internal audiences.
- FastAPI gives validation, versioned routes, and generated documentation for free — and those defaults implement Chapter 43's traceability requirements.
- The four product habits: versioned-together artefacts, request logging (your drift dataset), documented contracts, graceful degradation.
- Deployment is a six-line container plus a CI smoke test; where it runs is a cloud economics decision.
- The last rung is a maintenance contract — size it to the audience, and let the boring tool win when the boring tool suffices.

Practice Lab

1. Build the churn explorer on your Part V scores; add a segment filter and a "download campaign CSV" button; deploy it (Streamlit sharing or a container) and send the link to a non-analyst for a critique.
2. Wrap your trained pipeline in the FastAPI scorer; call it with curl; then send an invalid feature value and read the 422 — write two sentences on why this protected your model.
3. Add `/v2/churn` with a second model; route 10% of traffic to it via a simple flag; log which version scored each request; you have built a minimal champion-challenger.
4. Break it on purpose: corrupt the model file and restart; implement the graceful fallback (default score + warning header); document the behaviour in the API's human page.
5. Write the rung justification memo for three artefacts your organisation needs — one per rung 2, 3, and 5 — with audience, frequency, and maintenance cost for each.
6. The ship-week rubric: score your own capstone artefact against Section 49.4's four habits; fix the lowest score before calling any project done.

Further Reading

- Streamlit and FastAPI official documentation (both free, both excellent)
- *Designing Machine Learning Systems* — Huyen, chapters 7–8 (serving and products)
- Chapter 43 for the monitoring this chapter's logs feed.

Chapter 50: Teaching Analytics — A Course Design and Instructor's Guide

Part IX — The Extended Curriculum: Special Topics and the Craft of Teaching

"If you cannot teach it simply, you do not understand it well enough. If you can teach it, you own it forever."

In this chapter you will learn

- How to turn this book into a 14-week course — sequencing, pacing, assessment, and outcomes.
- The lecture-lab-feedback rhythm that keeps a technical course alive.
- How to grade analytics work honestly: rubrics for code, memos, and presentations.
- How to run studio sessions, project reviews, and a capstone showcase.
- How to teach the tools you already know — SQL, Excel, Python, Power BI, Stata, SPSS — as one discipline, not six.
- How to keep your own teaching current without drowning in tools.

50.1 Why Teaching Is the Best Way to Learn

The Feynman technique is famous for a reason. When you teach a concept, you are forced to retrieve it, organise it, confront its edge cases, and answer for its assumptions. Every "dumb question" from a student is a free audit of your understanding.

From Your Toolkit — SPSS: You have already taught, even if you have not called it that. Walking a manager through a crosstab in SPSS, explaining a chi-square result, defending a regression table in a meeting — these are teaching events. This chapter scales that instinct into a discipline.

This chapter is written for three audiences at once: the instructor building a formal course, the analyst running internal upskilling, and the self-learner who needs a syllabus. The mechanics are the same; only the venue changes.

50.2 Course Outcomes: Design Backwards

Before pacing or slides, answer: **what will students be able to do at the end?** Write outcomes as observable behaviours, not topics covered.

Strong outcomes for this course:

1. Frame a business question as an analytics question with a stated decision, metric, and baseline.
2. Build a trustworthy dataset from messy sources — joins, cleaning, validation — and document its lineage.

3. Choose and defend a statistical test or model, including its assumptions and limitations.
4. Build a dashboard or data product that a non-analyst can use unaided.
5. Communicate findings as a decision recommendation with quantified uncertainty.
6. Audit their own work: leakage checks, sensitivity analyses, and honest error accounting.

Every lecture, lab, and assessment maps to at least one outcome. If a session supports none, cut it. That is the discipline of backwards design: outcomes → assessments → activities → content. Most courses are built in the opposite direction and feel like it.

50.3 Sequencing: Follow the Book's Arc

The course follows the book's structure, which was designed as a syllabus:

- **Part I (weeks 1-2):** Foundations. Questions before data; data maturity; the Vs; data types; the six-tool bridge.
- **Part II (weeks 3-5):** Data engineering. Joins, grain, pipelines, warehousing, pandas-to-SQL, cleaning, quality.
- **Part III (weeks 6-8):** Statistics. Descriptives, probability, testing, sampling, regression.
- **Part IV (weeks 8-9):** Visualization and BI. Craft, metric trees, dashboards, storytelling.
- **Part V (weeks 10-12):** Machine learning foundations. Framing, leakage, baselines, models, evaluation, ethics.
- **Part VI (weeks 13-14):** Advanced ML — clustering, recommenders, NLP, deep learning, LLMs — taught selectively for a first course.

A 14-week term covers Parts I-VI in depth and Parts VII-VIII by immersion: governance, Spark, and careers slot into the capstone fortnight, where they belong (you teach pipelines best when students are suffering a broken one). In a two-semester design, the second semester runs Part IX's special topics (Chapters 46-50), Part X's ten industry playbooks as case-method seminars, Part XI's frontier chapters as honours reading, and Part XII's delivery craft as role-play. Teach eighty chapter/appendix units as a full-year course: the arc spans Parts I-XII, from first principles to the business of the craft, and it is coherent end to end.

Teaching Tip — The bridge is the course. Students arrive knowing tools — SPSS from a stats service course, Excel from an internship, SQL from a bootcamp. The course's job is to connect those tools into one discipline. Never say "forget Excel"; say "here is what Excel was doing all along, and here is where it runs out." Respect for prior knowledge is not politeness; it is pedagogy.

50.4 The Weekly Rhythm: Lecture, Lab, Feedback

A week that works:

Slot	Length	Activity	Failure mode it prevents
Lecture	60–90 min	Concepts, live demos, toolkit bridges	Pure theory with no grip
Lab	90–120 min	Students work the chapter's Practice Lab in pairs	Watching demos without doing
Feedback	15–30 min	Instructor reviews 2–3 student artefacts live	Students repeating errors all term
Memo	Homework	One-page write-up of the lab's finding	Analysis that cannot be communicated

The feedback slot is the engine. Reviewing real student work publicly — kindly, specifically — teaches more than any slide, because every student recognises their own errors in someone else's artefact. Rotate whose work is reviewed so everyone is showcased and everyone is polished.

Teaching Tip — Live coding beats slides. Type the SQL. Make the syntax error. Debug it. Students learn that errors are the job, not the exception. Prepare a backup notebook for when the live demo truly dies, but let small failures happen on purpose: the recovery is the lesson.

50.5 Pacing: Where Students Struggle

The known pain points, and the responses that work:

Concept	The struggle	The response
Joins and grain (Ch. 6–8)	Fan-out feels arbitrary until it bites	The duplicated-orders lab: let them find the 3x revenue inflation themselves
Hypothesis testing (Ch. 14)	p-value as ritual, not reasoning	The coin-flip simulation: see 5% of fair coins "significant" at n=20
Leakage (Ch. 22)	Everyone nods, everyone leaks	The leaky-feature audit: give them a dataset with a planted leak
Baselines (Ch. 23)	Skipping straight to XGBoost	The league table: naive baseline first, model second, side by side
Interpretation (Ch. 25, 33)	Reading coefficients as causes	The confounded coffee example: same data, two stories

Budget extra weeks for joins and testing; steal them from dashboards and deep learning if you must. A student who can grain a table and read a p-value honestly can learn Plotly next week. The reverse is not true.

The second semester's pain points are different: case-method seminars (Part X) struggle until students stop summarising and start deciding ("what would you do Monday morning?"); frontier chapters (Part XI) struggle when taught as surveys — one chapter, one paper, one replication keeps depth honest; and the delivery role-plays (Part XII) feel theatrical until the first hostile-question round lands, after which nobody mocks them again.

50.6 Assessment: Grade the Artefacts

Grade what analysts produce, not what exams can measure. A scheme that has survived many cohorts:

Component	Weight	Artefact	Graded on
Weekly memos	25 %	One-page lab write-up	Decision framing, honest uncertainty, clarity
Midterm project	25 %	Cleaned dataset + EDA memo	Lineage, validation, quality checks
Capstone	40 %	End-to-end analysis + presentation	Outcome rubric below
Participation	10 %	Labs, reviews, showcase	Presence and contribution

Exam questions live in Appendix H — use them for the viva component, where students defend their capstone choices aloud. Oral defence is the only assessment that reliably catches borrowed work and missing understanding simultaneously.

The capstone rubric, shared with students on day one:

Criterion	Weight	Excellent looks like
Framing	20%	Decision question stated; stakeholder named; success metric chosen and defended
Data craft	20%	Lineage documented; validation gates; no leakage
Analysis	25%	Baseline → model → comparison; assumptions checked; sensitivity run
Communication	25%	Memo a manager reads; charts follow Ch. 17; executive summary passes the stranger test
Reproducibility	10%	A classmate can rerun it from the repo

For honours/second-semester students, extend the capstone with one Part X playbook applied to a local organisation, one Part XI method replicated on public data, or one Part XII engagement role-play — one extension, done deeply, beats three done thinly.

50.7 Running the Studio: Reviews and Showcases

Project reviews. At weeks 4, 8, and 12, students present work-in-progress for ten minutes and receive structured critique: two strengths, two risks, one recommendation. This mirrors real practice (Ch. 72's engagement reviews) and teaches the hardest professional skill — receiving critique without defending.

The showcase. The final session is public: students present capstones to a real audience — managers from local businesses, other faculty, family. Real audiences raise the stakes productively; students polish slides that would have stayed rough for classmates. Invite a working analyst to give one sentence of feedback per presenter: practitioners' praise is remembered for years.

From Your Toolkit — Power BI: the showcase is a dashboard demo, so coach it as one: one screen, one message, thirty seconds to the point. Students who have presented a Power BI report to a manager already know the genre; name that knowledge.

Slide kits for every part. Appendix I holds a starter slide deck for each of the twelve parts — twelve decks, 8–12 slides each, with the one diagram that anchors each part: the analytics question ladder (Part I), the star schema (Part II), the testing logic tree (Part III), the chart-choice matrix (Part IV), the modelling loop (Part V), the represent-learn-predict map (Part VI), the scale decision test (Part VII), the insight-to-impact chain (Part VIII), the teach-back cycle (Part IX), the playbook canvas (Part X), the frontier map (Part XI), and the engagement arc (Part XII). Adapt them; do not merely display them. If you prepare eighty chapter maps from the deck skeletons you will never wonder what to say.

50.8 Teaching the Tools as One Discipline

The six modules of your foundation are not six subjects. Taught as one discipline, they become a grammar:

- **SQL** is the language of *what happened* — selection, aggregation, joins, windows.
- **Excel** is the language of *business arithmetic* — the model of the business everyone shares.
- **Python** is the language of *anything, at any scale* — the glue that connects every other tool.
- **Power BI** is the language of *what everyone sees* — the interface between analysis and organisation.
- **Stata** is the language of *econometric caution* — the tool that asks "identified by what?"
- **SPSS** is the language of *classical statistics* — the hypothesis-testing ritual, done properly.

When you teach joins, show them in SQL, then Excel-VLOOKUP, then pandas-merge, then the Power BI relationship model — same concept, four syntaxes. When you teach testing, show SPSS's crosstab, Stata's `prtest`, and the Python simulation of the same question. Transfer, not coverage: students who see one idea in four tools stop believing tools are knowledge.

Teaching Tip — Retire the syllabus, not yourself. Tools churn; the grammar does not. When a student asks about a tool this book does not cover, the honest answer is: "Here is the grammar it implements; find which parts it changes." Model that answer and you have taught Chapter 47's final lesson — how to stay current — as a living practice.

50.9 Your Own Development as an Instructor

Three habits keep a course alive:

1. **Teach one new thing every term** — a new dataset, a new method, a new failure story from practice. The course that never changes is the course that dies.

2. **Keep a teaching log** — what worked, what died, the questions that stumped you. Chapter 45's career habit, applied to the classroom.
3. **Bring practice in** — guest practitioners, your own consulting war stories, the Appendices E and J case portfolio. Students can smell the difference between a course about analytics and a course from analytics.

50.10 A Closing Note from the Author

If you have taught through this book, you have given eighty chapters of grammar, practice, and judgement to people who will use them in ways you will never see. That is the compounding of teaching: every analyst you train multiplies the honesty and the craft of every decision they touch. This chapter closes the taught syllabus, but not the book: Part X walks the industries where your students will work, Part XI pushes to the frontier they will extend, and Part XII teaches the delivery craft they will practise. Teach on.

— *Tatenda Makuvaza, "The Big Data Analyst"*

Key Takeaways

- Design backwards: outcomes first, assessments second, activities third. If a session maps to no outcome, cut it.
- The weekly rhythm is lecture → lab → live feedback → memo; the feedback slot is the engine.
- Grade artefacts, not exams: memos, projects, defences. The viva catches what written work hides.
- Students struggle predictably — at joins, testing, leakage, baselines, and interpretation. Budget time there, and let the pain points teach.
- Teach the six tools as one grammar, shown in four syntaxes. Transfer, not coverage.
- Teach one new thing every term, keep a log, bring practice in — a course is a product with users, and it deserves the same craft.

Practice Lab

1. Write the six course outcomes for your own context (your team, your classroom, your self-study) as observable behaviours — then audit this book's chapters against them: which chapters serve which outcomes?
2. Design your 14-week pacing grid: chapters to weeks, labs to chapters, one review week, one showcase week. Mark the two weeks you expect joins and hypothesis testing to eat.
3. Draft the capstone rubric adapted to your audience, and write the one-page handout version students receive on day one.
4. Run one studio review — a colleague or classmate presents work-in-progress for ten minutes; you deliver two strengths, two risks, one recommendation; write down what the format caught that informal feedback would not.

5. Prepare a ten-minute live-coded lesson on one concept from this book, deliberately including one recoverable error; teach it, then log what the error taught.
6. Take the Part XII hostile-question protocol and run it against your own capstone: present to a friend who must ask the three hardest questions from Chapter 74. Survive, then repeat.

Further Reading

- *Understanding by Design* — Wiggins and McTighe (backwards design)
- *Teaching Naked* — José Antonio Bowen (technology outside the classroom)
- Appendix H (assessment bank), Appendix I (slide kits), Appendix J (case portfolio)

PART X

Analytics Across Industries: Domain Playbooks

Chapters 51 - 60

Chapter 51: Retail and E-Commerce — The Ndineka Fashion House Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Retail analytics is one question in four costumes: what to stock, at what price, for whom, and how much is already dead."

In this chapter you will learn

- The retail playbook canvas: six slots that organise every retail analytics problem.
- Market-basket analysis: affinity, lift, and the difference between "bought together" and "buys more because".
- Markdown optimisation and price elasticity, honestly bounded.
- Customer lifetime value with the RFM bridge from your toolkit.
- The failure modes: the average customer, the dead-stock spiral, promo dependence.

51.1 The Retail Question

Ndineka Fashion House is a fictional Harare retailer: four stores, a growing online channel, 30,000 SKUs of seasonal clothing, and a founder — Ndineka herself — who describes her business in one sentence: "I buy six months before I sell, and I am wrong on purpose because being exactly right is impossible." That sentence is the entire retail problem. Inventory is frozen cash; the buy decision is made under uncertainty about weather, taste, and the economy; and the analyst's job is to shrink the wrongness, not to pretend it away.

Every retail analytics question reduces to one of four: **what to stock** (assortment), **at what price** (pricing and markdown), **for whom** (customers), and **how much is already dead** (inventory health). The playbook canvas organises them:

Slot	Ndineka's answer
Core question	Which lines, at what price, for which customers, bought how deep?
Unit of analysis	The SKU × store × week (and the customer, for the CLV layer)
Key metrics	Sell-through, gross margin, weeks of cover, dead-stock value, CLV
Data reality	POS transactions, seasonal buys, returns, a loyalty app, no true cost of returns
First project	The dead-stock ledger: name it, age it, price it
Failure mode	Promo dependence — margin spent to buy revenue

51.2 The Dead-Stock Ledger First

The highest-return first project in retail is almost always an inventory health report, because it requires no modelling and funds everything that follows. The discipline is aging: every SKU-store row gets weeks-on-shelf, sell-through rate, and a classification.

```
SELECT sku, store,
       weeks_on_shelf,
       units_sold / NULLIF(units_received, 0) AS sell_through,
       CASE
         WHEN weeks_on_shelf >= 26 AND sell_through < 0.30 THEN 'dead'
         WHEN weeks_on_shelf >= 12 AND sell_through < 0.50 THEN 'slow'
         ELSE 'healthy'
       END AS stock_class,
       retail_price * units_on_hand AS frozen_value
FROM inventory_snapshot
ORDER BY frozen_value DESC;
```

The `frozen_value` column is the meeting: when the leadership team sees that 22% of working capital is wearing a "dead" tag, markdown optimisation stops being a data science conversation and becomes a Tuesday agenda item.

51.3 Market-Basket Analysis

Baskets are the atom of retail behaviour. The classic measures, on the Soulfya's data you already know from Part II:

- **Support** — the share of baskets containing the pair. Rare pairs are noise.
- **Confidence** — given the anchor, how often the companion appears.
- **Lift** — confidence divided by the companion's base rate. Lift above 1 means the anchor *changes* the companion's probability.

```
pairs = (baskets.groupby(["anchor", "companion"])["basket_id"].nunique()
        .rename("both").reset_index())
pairs = pairs.merge(anchor_totals, on="anchor").merge(comp_totals, on="companion")
pairs["confidence"] = pairs["both"] / pairs["anchor_total"]
pairs["lift"] = pairs["confidence"] / (pairs["companion_total"] / n_baskets)
strong = pairs[(pairs["support"] >= 0.005) & (pairs["lift"] >= 1.5)]
```

The trap is reading lift as causation. Sadza-and-soft-drink has lift 1.4 because dinner is dinner — the pair reflects a meal occasion, not a promotion opportunity. The honest reading of basket analysis is *hypothesis generation*: every strong pair is a candidate for an experiment (Chapter 61's toolkit), never a layout change on its own.

51.4 Markdown and Price Elasticity

Markdown is not discounting; it is *managing the decay curve of a perishable asset* — fashion is perishable milk with better lighting. The analyst's contribution is elasticity bounded honestly. Fit demand (units sold) against price across comparable lines and weeks, but never as raw regression: confounders (season, stock depth, weather) ride along with every price cut. The defensible version is difference-based: compare similar items marked down at different times, or the same item across stores with staggered markdowns, and bound the elasticity estimate — "between -1.2 and -2.0" is an honest answer; "-1.61" is a costume.

Markdown optimisation, one paragraph: for each dead SKU, choose the smallest discount that clears stock before its terminal date (season end), given the elasticity bound and the salvage value. A -30% markdown that clears 80% of units beats a -50% markdown that clears 95% — the remaining 15% is worth more in the bargain bin than the margin given away on the first 80.

51.5 The Customer Layer: RFM to CLV

From Your Toolkit — Excel and SQL: you have built an RFM table before, even if you called it "sorting customers". Recency, frequency, monetary value is a pivot table in Excel (GROUP BY in SQL): every customer scored 1–5 on each axis, segments read off the grid — champions, at-risk, hibernating, gone. The bridge to CLV is Chapter 52's subject in full, but the retail version in one line: **CLV = margin per period × expected periods remaining**, with expected periods estimated from the retention curve of the customer's segment. Ndineka's champions (top RFM decile) generate 41% of margin — the number that justifies the loyalty programme's entire budget, and the number to check monthly.

51.6 Returns and the Long Tail

Two quiet killers. **Returns:** the online channel's return rate (Ndineka's is 28%, driven by sizing) is a data problem — fit notes, size charts, and the returns ledger joined back to product metadata turns "free shipping both ways" from a cost into a feedback loop that fixes the catalogue. **The long tail:** 30,000 SKUs means most SKUs sell rarely; models fit on averages will misprice the tail. Segment the assortment (core / seasonal / tail) and model each with its own honesty — the tail gets rules and monitoring, not deep learning.

51.7 Failure Modes

- **The average customer** — no one is average; averages misprice the tail and mis-target promotions. Segment first (Chapter 30), always.
- **The dead-stock spiral** — refusing to mark down because "the margin loss is visible and the cash release isn't". Show the ledger in cash terms.
- **Promo dependence** — every promotion trains customers to wait (Chapter 32's sleeping dogs, in retail form). Track full-price sell-through as a health metric, not just revenue.

- **Basket-lift layouts** — rearranging stores on untested affinities; Chapter 61's discipline applies.

Teaching Tip — The frozen-cash walk: take the class to any clothing store (or its website) and have each student pick three items, guess weeks-on-shelf and the coming markdown, and write the retailer's likely reason for the buy. Back in the lab, the dead-stock ledger feels like a familiar object, not a SQL exercise. The best retail exam question is one sentence: "Why is retail inventory perishable?"

Key Takeaways

- Retail is four questions: assortment, price, customers, inventory health — and the canvas keeps them straight.
- The dead-stock ledger is the highest-ROI first project: no models, immediate cash conversation.
- Basket analysis (support, confidence, lift) is hypothesis generation; lift is not causation, and layouts deserve experiments.
- Elasticity is a bound, not a number; markdown is perishable-asset decay management.
- RFM is the bridge from transactions to customers, and full-price sell-through guards against promo dependence.

Practice Lab

1. Build the dead-stock ledger on Soulfya's inventory snapshot (Appendix F generator): classify SKUs, sum frozen value, and write the one-page memo that makes the cash visible.
2. Compute support, confidence, and lift for all item pairs in a month of baskets; list the top ten by lift and annotate each with "occasion, substitution, or cause?" — then design the experiment that would settle one of them.
3. Estimate the elasticity bound for a markdown family using staggered-store prices; write the confidence interval and one paragraph on what confounds remain.
4. Build the RFM grid for Soulfya's Soul Circle members; identify the at-risk segment and size the win-back budget with an expected-value calculation.
5. Audit the long tail: what share of SKUs sold fewer than three units last quarter, and what rule (not model) would you apply to them?
6. The Ndineka memo: three recommendations — one assortment, one pricing, one customer — each with its metric, its baseline, and its reversal condition.

Further Reading

- *The New Science of Retailing* — Fisher and Raman
- *Why We Buy* — Paco Underhill (the physical analogue of basket analysis)

- Chapter 32 (uplift), Chapter 61 (causal discipline for experiments), Chapter 62 (pricing under uncertainty)

Chapter 52: Marketing and Customer Analytics — The Zuva Mobile Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Marketing without analytics is spending; marketing with analytics is still spending, but you know which half."

In this chapter you will learn

- The marketing playbook canvas, on a telecom's honest data reality.
- Acquisition analytics: CAC by channel, cohort curves, and the paid-growth trap.
- Churn, ARPU, and the CLV engine — the trio that values every subscriber.
- Uplift modelling for campaigns, revisited at telco scale.
- Attribution honesty: why last-click is a story, not a measurement.
- Failure modes: ROAS vanity, frequency fatigue, and the segment that cannot be saved.

52.1 The Marketing Question

Zuva Mobile is a fictional mobile network operator: 900,000 prepaid subscribers, 60,000 postpaid, a youth-heavy brand, and a marketing department that spends \$2.4 million a year across above-the-line media, agent commissions, digital, and sponsorships. The CFO's question is the one every marketing analytics practice is built to answer: **which spending changes customer behaviour, and which merely accompanies it?**

The canvas:

Slot	Zuva Mobile's answer
Core question	Which spend acquires, retains, and grows customers at positive lifetime value?
Unit of analysis	The subscriber × week; the campaign × cell
Key metrics	CAC, churn rate, ARPU, CLV, campaign incremental lift
Data reality	CDRs, recharges, campaign exposure logs with gaps, agent self-reports, no clean exposure truth
First project	The cohort retention curve by acquisition channel
Failure mode	Buying revenue with promotions and calling it growth

52.2 Acquisition: CAC and the Cohort Curve

Cost per acquisition by channel is arithmetic; it becomes analysis when paired with what happens after acquisition. The cohort curve is the method (Chapter 23's discipline applied to growth): group subscribers by acquisition month and channel, then plot retention and ARPU forward.

```
SELECT acquisition_cohort, channel,
       DATE_DIFF('week', acquisition_date, activity_week) AS weeks_in,
       COUNT(DISTINCT subscriber_id) AS active,
       ROUND(SUM(recharge_amount) / COUNT(DISTINCT subscriber_id), 2) AS arpu
FROM subscriber_activity
GROUP BY 1, 2, 3
ORDER BY 1, 2, 3;
```

The curve exposes what CAC alone hides: agent-acquired subscribers cost \$3.10 each and half are gone in six weeks; digital-acquired cost \$7.80 and hold 71% at week 26. **CAC without the cohort curve is a price tag on a mystery.** The channel that looks expensive per head is often the cheap one per *retained* head — and the metric that settles it is CAC amortised over surviving subscribers, or bluntly, payback weeks: months until cumulative margin covers acquisition cost.

52.3 The CLV Engine

The three numbers that value a subscriber base — churn, ARPU, CLV — form one engine:

- **Churn** (Chapter 27's survival view is the honest one): weekly active-churn for prepaid, where "active" must be *defined* (days since last recharge against the subscriber's own rhythm — a 30-day-absence means different things for a \$1-a-day user and a \$20-a-month user).
- **ARPU** (average revenue per user): revenue over subscriber-weeks, watched as a distribution, not an average — prepaid ARPU is bimodal.
- **CLV**: margin per period × expected periods remaining, where expected periods comes from the retention curve of the subscriber's segment, discounted if the horizon is long. The formula is one line; the craft is the segmentation that feeds it, because CLV computed on the whole base is a single number that describes no one.

From Your Toolkit — Power BI: the CLV engine is the classic three-page Power BI model: a cohort matrix visual (acquisition month × months-since, shaded by retention), a decomposition tree for ARPU, and a card row for payback weeks by channel. The toolkit bridge matters because the marketing team will *live* in this report; build it where they already work.

52.4 Campaigns: Uplift at Scale

Chapter 32 taught the four cells (persuadables, sure things, lost causes, sleeping dogs); telco scale makes the economics brutal. Zuva Mobile's airtime-bonus campaign reaches 300,000 subscribers per blast. Treating everyone wastes three-quarters of the spend and actively harms the sleeping dogs. The uplift discipline: run the campaign as a permanent 10% holdout (random, rotated monthly so no subscriber is forever untreated), model the *difference* in outcome between treated and untreated by segment, and target only segments whose uplift clears the cost line. The holdout is not lost revenue; it is the measurement budget, and it pays for itself the first time it kills a campaign that was buying revenue Zuva already had.

52.5 Attribution Honesty

Digital attribution is where marketing analytics goes to feel scientific. The honest position: **last-click attribution is a convention, not a measurement** — it assigns the conversion to the final touch, which systematically over-credits search and retargeting (the closers) and under-credits the media that created demand (the openers). The alternatives are all imperfect: multi-touch models are assumptions arranged in a funnel; media-mix modelling (Chapter 62's regression discipline at weekly granularity) is slow but honest about offline; and geo-experiments — splitting regions into treatment and control — are the closest to truth a big brand can buy. Zuva Mobile's practical stack: last-click for tactical optimisation within a channel, geo-experiments for the big budget questions, and the humility to say "unattributed" out loud.

52.6 Brand and Survey Analytics

Surveys are marketing's second dataset: aided/unaided awareness, consideration, and intent, sampled properly (Chapter 15) and tracked quarterly. The craft is refusing the vanity read: awareness that does not move consideration is not working, and intent questions need the stated-vs-revealed caveat. **From Your Toolkit — SPSS:** the tracking study is where your SPSS crosstab-and-chi-square training applies directly — significance tests on awareness by region and age band, with the sampling weights respected (Stata's `svy` discipline, same idea).

52.7 Failure Modes

- **ROAS vanity** — return on ad spend computed without a holdout: revenue that would have arrived anyway, booked as marketing's achievement.
- **Frequency fatigue** — reaching the same subscribers weekly until they churn to escape the campaign; watch opt-outs per exposure.
- **The unsaveable segment** — churn models that concentrate spend on the highest *risk* rather than the highest *uplift* (Chapter 32 again: some churners are lost causes; spend on the persuadables).
- **The average subscriber** — bimodal prepaid ARPU averaged into meaninglessness.

Teaching Tip — The holdout debate: stage it. Assign half the class to defend "every impression should be monetised, holdouts waste reach" and half to defend "unmeasured spend is unmanaged spend". Ten minutes, then show a real campaign's incremental lift beside its last-click ROAS. The gap between the two numbers is the whole section, seen rather than told.

Key Takeaways

- CAC is a price tag; the cohort retention curve is the product. Never quote one without the other.

- CLV is one formula over three honest inputs: segmented churn curves, ARPU as a distribution, and expected periods from the retention curve.
- Campaigns run on uplift with permanent holdouts — the holdout is the measurement budget, not lost revenue.
- Attribution conventions are not measurements: last-click for tactics, geo-experiments for truth, and "unattributed" as a respectable answer.
- Target uplift, not risk; watch frequency fatigue; and never average a bimodal base.

Practice Lab

1. Build the cohort retention matrix (acquisition month $\times$ weeks-in) for Zuva Mobile's data; identify the channel whose CAC looks cheapest per head and most expensive per survivor.
2. Compute payback weeks by channel; write the investment memo: which channel gets next year's incremental \$500,000, with the reversal condition stated.
3. Fit the CLV engine on segmented retention curves; produce the base valuation under three churn scenarios (base, +2 points, -2 points) and comment on the swing.
4. Design the uplift campaign: which segments get treated, what the holdout is, and what the success metric is before launch. Then compute the campaign's incremental profit from the (planted) results in Appendix F's campaign dataset.
5. Run the attribution audit: take a month of exposure and conversion logs, compute last-click ROAS, and list every assumption it makes in one table; mark which ones a geo-experiment would test.
6. The Zuva memo: the marketing budget is cut 15% — what do you cut, what do you protect, and what measurement survives the cut?

Further Reading

- *Marketing Analytics* — Wayne Winston (the operations-research view)
- Chapter 32 (uplift), Chapter 15 (sampling, for the survey layer), Chapter 62 (regression and causality for media mix)
- Zuva Mobile returns in Chapter 65 (Kumba's language services on Zuva's zero-rated bundle) and Appendix J

Chapter 53: Financial Services — The ZuvaPay Playbook

Part X — Analytics Across Industries: Domain Playbooks

"In lending, the model is not the product; the loss function is the product."

In this chapter you will learn

- The financial services canvas: risk, fraud, and regulation as one analytical system.
- Credit scoring the modern way, with the classic scorecard as its interpretable core.
- Class imbalance done honestly: cost-based thresholds for fraud.
- Explainability as law, not preference — and what a regulator actually asks.
- Population stability and drift monitoring as a *prudential* habit.
- Failure modes: score gaming, threshold drift, and the excluded applicant.

53.1 The Financial Services Question

ZuvaPay is a fictional payments-and-credit fintech: 1.1 million wallet users, 40,000 merchant points, and a fast-growing nano-loan product — \$10 to \$150, 30-day terms, approved in seconds from wallet behaviour. Its analytical life is dominated by three questions: **who will repay** (credit risk), **who is stealing** (fraud), and **can we prove it** (regulation). The twist that makes this playbook distinct from every other chapter in Part X: the *cost of being wrong is asymmetric and regulated* — a declined good customer costs \$4 of margin; a funded fraudster costs \$150 of principal; and a model the regulator cannot understand is a model you cannot deploy, whatever its AUC.

Slot	ZuvaPay's answer
Core question	Approve, price, and detect — at what thresholds, provable to whom?
Unit of analysis	The application (credit); the transaction (fraud)
Key metrics	Default rate, loss rate, approval rate, false-positive cost of fraud, model stability (PSI)
Data reality	Wallet histories, device signals, bureau thin-files for most applicants, adversarial fraud
First project	Vintage curves on the existing loan book
Failure mode	Accuracy that cannot survive an audit

53.2 Credit Risk: Vintage Curves First

Before any model, the vintage analysis: group loans by origination month, track cumulative default through days-past-due windows (30/60/90), and read the curves. The vintage chart is the lending business's ECG — it shows whether the book is improving cohort by cohort, and it calibrates everyone (credit committee included) to the actual default base rates before a single score exists.

The scoring stack is then a Chapter 23-25 problem with domain clothing: features from wallet history (top-up rhythm, spend volatility, airtime borrowing, merchant mix), a binary target (90+ days past due within term), and a *business* metric — profit at threshold, not AUC. The classic scorecard remains the deployed core in most regulated lenders, and its craft is worth knowing:

```
# Weight-of-evidence binning: the scorecard's interpretability engine
woe = (df.groupby("bin")["target"].agg(["sum", "count"]))
woe["good"] = woe["count"] - woe["sum"]
woe["woe"] = np.log(
    (woe["good"] / max(woe["good"].sum(), 1))
    / (woe["sum"] / max(woe["sum"].sum(), 1))
)
```

Each feature becomes a small table of bins and weights; the score is an addition; and "your recharge regularity contributed -12 points" is an explanation a call-centre agent can read aloud. **From Your Toolkit — SPSS:** this is the direct descendant of the SPSS scorecard tradition you met in training — the same binning logic, now fitted in Python and monitored like a system.

53.3 Fraud: Imbalance and the Cost Line

Fraud detection is the hardest imbalance problem in practice: 0.3% fraud prevalence, an adversary who adapts, and a threshold that is a *business decision* wearing a technical costume. The honest method:

1. Baseline rules first (velocity, device reuse, geo-inconsistency) — they catch the blunt fraud and they are explainable.
2. A model for the residue (gradient boosting on behavioural sequence features), evaluated on **precision at the review-queue size** the operations team can actually staff, not on accuracy or even AUC.
3. The threshold set from the cost matrix: review is worth it when $\text{expected fraud caught} \times \text{loss size} > \text{review cost} \times \text{false-positive rate} \times \text{queue size}$. At ZuvaPay's numbers (\$150 average loss, \$2.20 per review, 30 minutes of customer friction), the queue that clears this line is 1.8% of transactions — and *that* number, not the model, is the deliverable.

53.4 Explainability as Law

The regulatory layer is not optional garnish. A ZuvaPay decline triggers, in most markets, a right to explanation: the principal reasons, in human language, in a regulated format. The practical stack is Chapter 33's: reason codes from the scorecard bins (native), SHAP-style attributions for the model layer (Chapter 33), and a documented mapping from every feature to its data lineage (Chapter 42's governance, enforced by the regulator rather than chosen by taste). The discipline to internalise: **explanation artefacts are part of the model artefact**, versioned together, tested together — an explanation that lags the model is a compliance incident waiting for its auditor.

53.5 Monitoring: PSI and the Champion-Challenger Rhythm

Credit models decay as the population and the economy move. The standard vigilance is the **population stability index** on every feature and the score itself, monthly:

```
SELECT score_band,
       COUNT(*) * 1.0 / SUM(COUNT(*)) OVER () AS share_now,
       expected_share,
       ROUND((share_now - expected_share)
            * LN(share_now / NULLIF(expected_share, 0)), 4) AS psi_part
FROM scored_applications
GROUP BY score_band, expected_share;
```

PSI above 0.25 on a feature or the score is the alarm: the world the model learned has moved. The response is Chapter 43's champion-challenger: refit as challenger, shadow-score live, swap only when the challenger beats the champion on recent data — with the swap itself documented, because the regulator will ask.

53.6 Failure Modes

- **Score gaming** — applicants (and agents) reverse-engineer the scorecard and manufacture the signals; monitor feature distributions for spikes at bin edges.
- **Threshold drift** — the model is stable, the queue is not: a review queue allowed to grow silently changes the product's economics and its customer experience.
- **The excluded applicant** — thin-file applicants score low by absence of data, not absence of worth; the fair-lending audit (Chapter 29's disparate-impact discipline) is a prudential duty, and alternative-data features must justify themselves against it.
- **Accuracy without auditability** — the 0.87-AUC deep model that cannot ship; in regulated credit, the explainable 0.84 wins the right to exist.

Teaching Tip — The decline letter: give students a scored applicant (features, bins, points) and have them write the regulated decline letter — principal reasons, plain language, the applicant's rights. Nothing exposes a shallow understanding of a scorecard faster than trying to explain one to the person it rejected. Pair it with the mirror exercise: the letter the *falsely declined* applicant deserves.

Key Takeaways

- Lending analytics starts with vintage curves, not models — the book's ECG before its engine.
- The scorecard's binning-and-addition design is not nostalgia; it is explainability engineered into the artefact.
- Fraud thresholds are business decisions: staff the queue from the cost line, and evaluate on precision-at-queue-size.
- Explanations are part of the model artefact — versioned, tested, regulated.
- PSI monthly, champion-challenger always; and the fair-lending audit is not optional.

Practice Lab

1. Build vintage curves on ZuvaPay's loan book (Appendix F generator); identify the deteriorating cohort months and hypothesise (with evidence) why.
2. Fit a scorecard: WOE-bin five features, fit a logistic regression on the WOEs, produce points, and write the decline letter for three named applicants.
3. Set the fraud queue: given the cost matrix and the model's precision-recall curve, choose the queue size, defend it in dollars, and show what the queue costs in customer friction.
4. Compute monthly PSI on the score and top features; find the feature that broke 0.25 and write the incident note.
5. Run the fair-lending audit: approval rates and score distributions by applicant segment (urban/rural, age band); document any gap and propose the investigation.
6. The credit-committee memo: raise approval rate 5 points without raising loss rate — what lever do you pull, what would prove you wrong, and what does the regulator need to see?

Further Reading

- *Credit Risk Scorecards* — Siddiqi (the scorecard bible)
- Chapter 23 (evaluation and cost), Chapter 29 (fairness), Chapter 33 (explanation), Chapter 43 (monitoring)
- Appendix J Case 2 for the leakage post-mortem pattern that applies to credit models verbatim

Chapter 54: Healthcare Analytics — The Batanai Health Trust Playbook

Part X — Analytics Across Industries: Domain Playbooks

"In healthcare the error costs are not symmetric, the ethics are not optional, and the chart is someone's mother."

In this chapter you will learn

- The healthcare canvas: outcomes, flow, and cost under a duty of care.
- Readmission risk: the canonical clinical prediction problem, done honestly.
- Patient flow and staffing: queueing and simulation for wards and clinics.
- Survey and epidemiology analytics with sampling weights respected.
- Privacy beyond compliance: consent, minimum necessary, and the harm test.
- Failure modes: the risk score as a gate, Goodhart on clinical targets, and the average patient.

54.1 The Healthcare Question

Batanai Health Trust is a fictional non-profit health network: three clinics and a 60-bed district hospital, 140,000 patients on file, donor-funded, and measured by the Ministry and the donors on a scorecard it did not choose. Its analytical questions: **who needs care before they return deteriorating** (risk), **how do we flow patients through scarce beds and staff** (operations), **what is actually happening in the population** (epidemiology), and **what can we honestly claim to our funders** (evaluation). The distinctive constraint is the duty of care: an error here is not a lost sale; it is a person.

Slot	Batanai's answer
Core question	Which patients need which intervention, when — and does it work?
Unit of analysis	The patient-episode; the clinic-day; the catchment sector
Key metrics	Readmission rate, time-to-treatment, bed occupancy, programme yield, cost per outcome
Data reality	An EMR with coding gaps, paper registers at two clinics, staff logs, DHIS2 reports to the Ministry
First project	The missed-appointment and readmission register, unified
Failure mode	A model that gates care instead of guiding it

54.2 The Unified Register First

The first project is deliberately unglamorous: one patient-level register unifying EMR extracts, the clinics' paper registers (photographed, keyed, and *validated* — Chapter 11's discipline, because a register nobody trusts is a rumour with columns), and follow-up call logs. The register's columns are the analytics agenda: patient key, episode dates, diagnoses (coded, with an "uncoded" bucket that is itself a finding), discharge destination, appointment dates, attendance, and outcomes. Every later model, dashboard, and donor claim stands on this register — which is why it is a governance artefact (Chapter 42) with a named steward, not a spreadsheet on someone's laptop.

54.3 Readmission Risk, Honestly

The canonical problem: predict 30-day readmission, so that discharge planning and follow-up can concentrate where risk is highest. The honest version has four features that distinguish it from the Kaggle costume:

1. **The baseline is clinical, not algorithmic.** Nurses already know the highest-risk patients by diagnosis, age, and social circumstance. The model must beat clinical judgement (or, more realistically, *augment* it by surfacing non-obvious risk — the young patient with a chaotic visit history).
2. **The target is defined by operations, not purity.** "Readmission to any facility within 30 days of discharge" — and the register must be able to see transfers, or the target is silently wrong.
3. **Calibration beats discrimination.** A risk score that says 20% must mean 20%, because the intervention (a follow-up call, a review slot) is rationed by threshold; miscalibrated risk rations it wrong (Chapter 25's calibration discipline, now clinical).
4. **The intervention defines the metric.** The point is not AUC; it is whether flagged patients who received the follow-up bundle readmit less — an uplift question (Chapter 32) wearing scrubs. Deploy with a randomised holdout of flagged patients from day one; it is both ethical and the only proof that works.

54.4 Flow, Beds, and Staffing

The operations layer is Chapter 66's queueing and Chapter 67's simulation, in a hospital's clothing. The ward is a queue: arrivals (admissions by hour and day), servers (beds and nurses), and a waiting discipline with clinical priority. Batanai's maternity ward runs at 92% peak occupancy — past the ρ -cliff where Chapter 66's waiting times explode — and the analyst's contribution is the queueing arithmetic that turns "we feel busy" into "at 92% utilisation your expected wait is four hours; at 80% it is ninety minutes; here is the staffing plan that buys the difference". The simulation then tests the plan against reality: elective-schedule changes, a flu week, one nurse absent — the p90 day, not the average day, is what the ward manager lives.

54.5 Population and Survey Analytics

The epidemiology layer answers "what is happening in the catchment": prevalence estimates from clinic data (biased toward the sick who attend — the classic selection problem, Chapter 15), corrected by community surveys with proper sampling. **From Your Toolkit — Stata:** this is Stata's home ground — `svyset` with the sampling design, `svy: proportion` for prevalence with design-corrected confidence intervals, and `svy: logistic` for the risk-factor models. The same discipline in Python (`statsmodels` with survey weights) gives the same answers; what must never happen is the unweighted crosstab presented as a population estimate — the single most common published error in programme reporting.

54.6 Privacy Beyond Compliance

Health data is the hardest privacy case: re-identification is easy (rare diagnosis + suburb + age = one person), consent was often gathered pre-analysis, and the harm from leakage is direct. The practitioner's floor, beyond every regulation: **minimum necessary** (the analysis extract carries only the columns the question needs — the full EMR never leaves the clinical system), **the harm test** (before any release, ask what the worst reader could do with it), and **aggregate-only reporting** for anything leaving the Trust — with small-cell suppression (no cell below 5) as the default rule. Chapter 68's formal privacy engineering is the frontier; this section is the floor beneath it.

54.7 Failure Modes

- **The risk score as a gate** — a model intended to *guide* discharge planning quietly becomes the rule that denies the follow-up call to the "low-risk"; monitor what decisions the score actually feeds.
- **Goodhart on clinical targets** — time-to-treatment targets met by a triage queue that starts *after* registration; every operational metric needs its companion audit (Chapter 45's discipline, clinically applied).
- **The average patient** — clinical subgroups (age bands, comorbidities, distance) differ enough that one model or one threshold mis-serves them; stratify evaluation, not just training.
- **The donor-pleasing estimate** — unweighted clinic data reported as population prevalence; the error of Section 54.5, published.

Teaching Tip — The chart is someone's mother: run the ethics round before the lab. Give students a de-identified extract and the local newspaper's court report page; thirty minutes to attempt re-identification of one patient. The exercise lands harder than any lecture on privacy — and it motivates small-cell suppression, minimum necessary, and aggregate-only reporting as *craft*, not compliance theatre.

Key Takeaways

- Start with the unified register: every model, dashboard, and donor claim stands on it; it is a governance artefact with a steward.
- Readmission models must beat or augment clinical judgement, calibrate (not just discriminate), and be evaluated as an intervention with a randomised holdout.
- Queueing arithmetic and simulation turn "we feel busy" into staffing plans tested against the p90 day.
- Survey analytics respects the design: unweighted clinic data is not population prevalence.
- The privacy floor is minimum necessary, the harm test, and small-cell suppression — Chapter 68 is the frontier, not the floor.
- Watch what decisions each score feeds: gates are the failure mode of guides.

Practice Lab

1. Build the unified register's validation report on the Batanai extract (Appendix F generator): duplicate patient keys, uncoded diagnoses, appointment dates after discharge, and the paper-register reconciliation table.
2. Fit the readmission model; evaluate against the clinical baseline, report calibration by decile, and write the deployment note that includes the randomised holdout design.
3. Queueing arithmetic: given arrival and service rates by shift, compute expected waits at current staffing and at two alternatives; present the staffing memo with the p90 simulation results beside the means.
4. The survey correction: estimate prevalence from clinic data, then re-estimate with the community survey's design weights; write the two sentences a donor report needs explaining the difference.
5. Run the re-identification exercise on your own extract; document the three easiest attacks and the specific controls that block each.
6. The Goodhart audit: pick one Ministry indicator, describe how it could be met while care worsens, and design the companion metric that catches it.

Further Reading

- *Clinical Prediction Models* — Steyerberg (the honest textbook of the genre)
- Chapter 32 (uplift as intervention evaluation), Chapter 63 (survival curves for time-to-readmission), Chapter 66–67 (queueing and simulation)
- Appendix J Case 4 for the policy-evaluation pattern that donor claims should follow

Chapter 55: Agriculture and Food Security — The Chikafu Foods and NGA Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Farming is a weather hedge wearing overalls; food security analytics is the hedge book."

In this chapter you will learn

- The agriculture canvas: yield, logistics, and markets as one food-system problem.
- Yield forecasting that combines agronomy, weather, and remote sensing — with honest error bars.
- Grain logistics and the buffer stock: seasonality, storage economics, and the import decision.
- Smallholder data: satellite plus surveys, and the bias in every convenient source.
- The co-operative evaluation pattern: Sable Fields and the missing-data discipline.
- Failure modes: the average farm, the planting-date fallacy, and precision without coverage.

55.1 The Food-System Question

Chikafu Foods is a fictional grain miller and food processor: it buys maize and small grains from 40,000 smallholders and three commercial estates, mills for retail, and holds a supply contract with the **National Grain Authority (NGA)** — the fictional state buyer that runs the strategic reserve. Their analytical questions chain together: **how much will grow** (yield forecast), **where and when will it move** (logistics), **what should we pay and stock** (markets and the buffer), and **what did the programmes actually do** (evaluation). Agriculture is the playbook where the *system* is the unit: a miller's forecast error becomes the NGA's storage decision becomes the consumer's price — and the analyst who sees only their own link optimises the wrong thing.

Slot	Chikafu / NGA's answer
Core question	How much grain, where, when — and what should the reserve do about it?
Unit of analysis	The field/plot for yield; the depot-week for logistics; the district-season for food security
Key metrics	Yield (t/ha), forecast error at lead time, depot stock weeks, price seasonality, food-security coverage
Data reality	Remote sensing pixels, sparse weather stations, farmer registers with staleness, market price boards
First project	The yield forecast with published, scored error — plus the error history
Failure mode	Confident averages: the mean yield that describes no farm and the reserve that trusts it

55.2 Yield Forecasting with Honest Error Bars

The yield forecast is the sector's headline model, and its craft is layering information sources with their uncertainties:

- **Agronomic structure first:** planted area $\times$ trend yield, adjusted for season start (planting dates are a strong signal — late onset means lower yield potential, and the adjustment is bounded by agronomy, not fit).
- **Weather on top:** rainfall accumulation and distribution by dekad (ten-day period) from the sparse station network plus gridded products; distribution matters — three well-spaced rains beat one deluge.
- **Remote sensing:** vegetation indices (NDVI-type) from public satellites give a canopy-health read per pixel, aggregated to district — the pixel sees every field, including the ones nobody reported.
- **Ground truth:** crop-cut surveys on sampled plots — small, expensive, and the only anchor that keeps the satellite honest.

The forecast is published with the *history of its errors*: "district maize yield, 1.9-2.3 t/ha, past-April error ± 0.4 ". A forecast without its scored error history is a rumour with decimals — and the NGA's storage decisions are exactly the kind of decision (Chapter 61's language) that deserves an interval, not a point.

```
# Layered yield forecast: agronomic base, weather adjustment, NDVI residual
base = planted_area * trend_yield           # agronomic anchor
weather_adj = 1.0 + (-0.35) * late_onset_frac # bounded by agronomy
ndvi_adj = 1.0 + 0.5 * (ndvi_z - ndvi_clim_z) # canopy anomaly
forecast = base * weather_adj * ndvi_adj
err = forecast - crop_cut_yield              # scored against ground truth
```

55.3 The Buffer Stock and the Logistics Layer

The NGA's reserve problem is seasonal by construction: buy at harvest when prices sag, release in the hungry months when prices spike, hold enough to smooth but not so much that storage and losses eat the budget. The analytics are Chapter 28's forecasting (price seasonality with its shocks), Chapter 47's optimisation (depot siting and movement minimisation — grain is heavy and fuel is dear), and one accounting discipline: **carryover cost per tonne per month** (storage + interest + shrinkage), because the release decision is "sell now versus hold", and that number is the whole trade. The failure to fear is the reserve that becomes a hoard: bought for price smoothing, held for prestige, financing itself out of existence.

55.4 Smallholder Data: Satellite and Survey

The sector's hardest data problem: the average farm is 1.5 hectares, transactions are informal, and every convenient source is biased. Farmer registers overcount active farmers (dead entries never leave); purchases undercount sales to informal traders; satellite sees canopy but not crop (maize and weeds are both green). The craft is triangulation with explicit bias accounting — satellite area as the coverage layer, register+survey as the behaviour layer, and the gap between them *reported* as a finding, not patched silently. **From Your Toolkit — Stata:** the crop-cut and household survey analysis is Stata territory — design-based estimates, panel regressions for programme effects, and the robust standard errors that keep small-sample honesty — with Chapter 15's missing-data discipline standing behind it.

55.5 The Evaluation Pattern: Sable Fields

The co-operative evaluation of Appendix J Case 5 (Sable Fields Co-operative: 2,400 farmers, a four-year productivity grant, and a year of missing records) is the sector's canonical evaluation shape: treatment that was not randomised, data with gaps that are informative, outcomes complicated by exit. The pattern — quantify the missingness, choose the strategy, bound the estimate, model the exits with survival curves (Chapter 63), and publish the gap in the first paragraph — is the discipline every agriculture programme report owes its funders. Agriculture is where evaluation honesty is most tested, because the funders want transformation stories and the data usually offers bridges.

55.6 Failure Modes

- **The average farm** — mean yield over a right-skewed distribution of 1.5-hectare plots; median and deciles are the sector's honest descriptors.
- **The planting-date fallacy** — late planting correlates with poorer farmers, so "late planters yield less" is partly poverty, not delay; the covariate structure must be respected before advising on timing.
- **Precision without coverage** — a beautiful model on the three commercial estates' clean data, extrapolated to 40,000 smallholders it never saw; the coverage audit belongs in every agriculture report.
- **The prestige reserve** — Section 55.3's hoard; the buffer stock needs its own dashboard, with carryover cost and release triggers, before it needs a forecast.

Teaching Tip — The two-map exercise: hand students two district maps side by side — satellite greenness and crop-cut yield — and ask what the differences mean. The discussion discovers every section of this chapter: coverage versus ground truth, canopy versus crop, and why the reserve needs both maps plus a survey. It is the whole playbook in one visual, and it teaches the humility of remote sensing better than any lecture.

Key Takeaways

- Agriculture is a system: yield, logistics, and markets chain together, and optimising one link in isolation mistimes the whole.
- Yield forecasts layer agronomy, weather, and remote sensing — and publish their scored error history or count as rumours.
- The buffer stock is a smoothing instrument with a carrying cost; give it triggers and a dashboard, not prestige.
- Every convenient smallholder source is biased; triangulate and report the gaps instead of patching them silently.
- Programme evaluations follow the Sable Fields pattern: missingness quantified, estimates bounded, exits modelled, honesty in the first paragraph.

Practice Lab

1. Build the layered district yield forecast on the season's data; score it against crop-cut plots, and write the release note with the interval and the error history attached.
2. Price seasonality: decompose five years of market maize prices into trend, seasonal, and shock components; recommend the reserve's buy window and the release trigger.
3. The carryover audit: compute storage, interest, and shrinkage per tonne per month for each depot; identify the depot where holding grain is destroying value.
4. Coverage audit: compare register-reported planted area against satellite-derived area by district; map the gap and write the two hypotheses it most plausibly supports.
5. Design the Sable Fields successor evaluation: a grant renewal where you control the design from day one — specify the randomisation (or its honest alternative), the measurement plan, and the pre-registered analysis.
6. The food-security memo: the forecast comes in 12% below trend — what does Chikafu do, what does the NGA do, and what does each need to know that the other holds?

Further Reading

- *Agricultural Production Economics* — the yield-response foundations
- Appendix J Case 5 (Sable Fields) for the full evaluation; Chapter 63 (survival for exits); Chapter 68 (privacy for farmer registers)
- FAO's seasonal forecast methodology papers — the best public example of error-scored forecasting

Chapter 56: Energy Analytics — The Sable Energy Playbook

Part X — Analytics Across Industries: Domain Playbooks

"An electricity system has no warehouse: every kilowatt is consumed the instant it is made, forever."

In this chapter you will learn

- The energy canvas: demand, supply, and money under a zero-inventory constraint.
- Load forecasting: the model that runs the company, from daily shape to tariff-sensitive peaks.
- Prepaid meters and collections analytics: revenue assurance as a first-class product.
- Losses: technical versus non-technical, and the anomaly discipline that tells them apart.
- Tariff and subsidy analysis with elasticity respected — the policy layer.
- Failure modes: the average day, the duck curve, and the meter that lies.

56.1 The Energy Question

Sable Energy is a fictional national distribution utility: 780,000 metered connections, a peak demand that has doubled in a decade, prepaid meters in most homes, and a generation mix it partly controls (two coal units, one hydro contract, growing solar) and partly prays over (rainfall for the hydro, evening for the solar). The defining constraint makes energy unique among this part's playbooks: **electricity cannot be stored at scale**, so supply must meet demand every minute — which makes forecasting not a reporting function but the operational spine. The analytical questions: **what will demand be** (load forecast), **is everyone paying** (collections and losses), **where is the power going that no one paid for** (loss analytics), and **what should it cost** (tariffs and subsidies).

Slot	Sable Energy's answer
Core question	How much power, when; who is paying; where are the losses?
Unit of analysis	The connection × 30-minute interval; the feeder-day; the tariff band
Key metrics	Peak demand (MW) and timing, forecast MAPE at day-ahead, collection ratio, loss % (technical vs nontechnical), load factor
Data reality	Smart-meter interval data (partial), prepaid vending logs, SCADA feeds, feeder sensors, theft and tamper records
First project	The day-ahead load forecast with a published error, by region
Failure mode	Planning the average day and being destroyed by the peak

56.2 Load Forecasting: The Model That Runs the Company

Demand is the most structured series in this book: strong daily shape (morning rise, evening peak), weekly shape (weekday/weekend), seasonal shape (winter heating, summer cooling), and event structure (paydays, public holidays, a national football final). The craft is treating those structures as features, then letting the model handle the residuals:

```
features = pd.DataFrame({
    "hour":      ts.index.hour,
    "dow":      ts.index.dayofweek,
    "is_weekend": (ts.index.dayofweek >= 5).astype(int),
    "temp_c":    weather.reindex(ts.index)["temp"],
    "payday_0to2": payday_proximity(ts.index),    # 0,1,2 days after payday
    "lag_1d":    ts["mw"].shift(24 * 2),          # same interval yesterday
    "lag_1w":    ts["mw"].shift(24 * 2 * 7),      # same interval last week
})
```

The evaluation is Chapter 28's seasonal-naïve discipline (the league table, always) and the metric that matters operationally: **day-ahead MAPE at the evening peak**, because a 3% average error that is 8% at 18:30 is a utility that buys emergency power at the worst prices. The forecast feeds the unit-commitment decision (which generators run tomorrow), so its error has a price tag — and Chapter 61's counterfactual language applies: the value of a better forecast is the avoided cost of the mistakes, computable and reportable.

56.3 The Prepaid Layer: Collections and Vending

Prepaid metering turns the revenue question inside-out: instead of billing after consumption, the utility collects *before* it — and analytics shifts from receivables to vending behaviour. The questions are behavioural and operational: purchase frequency and size by segment (a household's vending rhythm is its energy fingerprint), disconnection-and-reconnection patterns, and the *timing* load — paydays, school fees season, harvest income in rural feeders. **From Your Toolkit — Excel and Power BI:** the vending analytics layer is a classic Excel-to-Power-BI product: purchase registers aggregated to connection-week, a decomposition view for seasonality, and a collections dashboard the commercial team lives in. The distinctive energy lesson for the analyst: in prepaid systems, **demand data doubles as payment-capability data**, and the two must be read together — a feeder whose consumption falls while its vending rises is telling you something about metering, not weather.

56.4 Losses: Technical and Non-Technical

Losses are the gap between energy bought (or generated) and energy sold, and the analytics exist to split that gap in two: **technical losses** (physics — resistance in lines and transformers, computable from network models and loading) and **non-technical losses** (theft, tampering, metering error, billing gaps — the human share). The method is Chapter 46's anomaly discipline at feeder granularity:

- Energy balance per feeder-day: energy in (substation) versus energy vended plus estimated unmetered use — the residual is the loss.
- Profile the residuals: technical losses rise smoothly with load squared; tampering leaves step changes and consumption-collapse signatures that physics cannot explain.
- Rank and investigate: feeders ranked by unexplained residual per connection, crossed with tamper-inspection outcomes, produce the inspection schedule — and the *audit* of whether inspections are working.

The quiet trap: an aggressive loss-reduction campaign can chase the physics. The split — modelled technical loss versus residual — is what keeps the campaign aimed at the human share.

56.5 Tariffs, Subsidies, and the Elasticity Layer

The policy layer is where energy analytics meets Chapter 61 head-on: tariffs change behaviour, and a tariff modelled with zero elasticity is a plan for a world that does not exist. The pattern is Appendix J Case 4's (Harare Metro Water) transposed: cross-subsidy designs (lifeline blocks funded by higher bands) evaluated under elasticity estimated from historical tariff shocks, with revenue-neutrality checked as a *range*, never a point. Sable Energy's version: the lifeline block (first 50 kWh cheaper) whose true cost depends on how many high-band customers respond to their price rise by installing solar — the elasticity term, wearing panels.

56.6 Failure Modes

- **Planning the average day** — capacity is sized and committed against peaks; every average-based report understates the risk by construction.
- **The duck curve** — solar shifts net demand's shape (deep midday trough, steep evening ramp); a forecast trained on gross consumption miscommits the evening units exactly when they are dear.
- **The meter that lies** — smart-meter data is sensor data: gaps, drift, and stuck registers are findings to model (Chapter 11's validation gates, per interval), not background noise to average through.
- **Loss theatre** — reporting total losses without the technical split, then celebrating reductions that were physics all along.

Teaching Tip — The zero-inventory hour: open the class by asking what a warehouse is for, then deleting it from the electricity system. Everything in the chapter follows from that one absence: forecasting becomes operations, storage becomes the most expensive product on earth (batteries), and the evening peak becomes the moment the company lives or dies. Students who feel the constraint stop treating load forecasting as "just a time series".

Key Takeaways

- Electricity's zero-inventory constraint makes the load forecast the operational spine: evaluate at day-ahead peak MAPE, and price the forecast's errors.
- Prepaid systems shift analytics from receivables to vending behaviour — and consumption data doubles as payment-capability data.
- Loss analytics exists to split physics from people: model technical loss, then investigate the residual, and audit the investigations.
- Tariff and subsidy analysis respects elasticity — revenue neutrality is a range, and the lifeline block has a solar-panel term.
- Every average is a trap: peaks, ramps, and p90 days are the utility's real calendar.

Practice Lab

1. Build the day-ahead load forecast for one region (Appendix F generator supplies interval demand and weather); run the league table including seasonal-naïve, and report MAPE overall and at the evening peak.
2. Price the forecast: given balancing costs, translate the forecast error distribution into an expected cost, and the 2-point MAPE improvement into its avoided-cost value.
3. The vending layer: build connection-week vending aggregates, find the payday signature, and identify three feeders whose consumption-vending divergence suggests metering problems.
4. The loss split: compute feeder-day residuals, estimate the technical loss model, rank feeders by unexplained residual per connection, and design the inspection experiment that tests whether the ranking works.
5. The tariff memo: evaluate the lifeline design under elasticity bounds from the last tariff shock; show the revenue range and name the political question the range hides.
6. The resilience test: simulate the p10 hydro-rainfall year against your demand forecast — what breaks first, and what would you pre-negotiate?

Further Reading

- *Electricity Economics* — the tariff and peak-pricing foundations
- Chapter 28 (forecasting), Chapter 46 (anomaly detection for losses), Chapter 61 (elasticity and counterfactuals)
- Appendix J Case 4 for the cross-subsidy evaluation pattern this chapter borrows

Chapter 57: Government and Public Sector Analytics — The Harare Metro Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Public data answers to everyone at once: the minister, the auditor, the taxpayer, and the person in the queue."

In this chapter you will learn

- The public-sector canvas: services, evidence, and scrutiny under permanent visibility.
- Programme evaluation culture: the DiD discipline and the null result that must be published.
- Operational analytics for public services: queues, registers, and the trust problem.
- Open data and privacy: publication as an engineering problem.
- Procurement analytics: the anomaly layer where public money leaks.
- Failure modes: Goodhart's law at national scale and the report nobody reads.

57.1 The Public-Sector Question

Harare Metro is the fictional metropolitan authority you met in Appendix J Case 4 (the water tariff) — but its analytical life is broader than one tariff: refuse collection, water and sanitation, road maintenance, licensing, and the social programmes it administers on behalf of national government. The public-sector pattern differs from every other playbook in Part X on three axes: **many masters** (the political principal changes, the permanent service endures, the public owns the data), **full scrutiny** (an auditor, a journalist, or an opposition councillor may read any number you publish), and **the null-result duty** (programmes must be evaluated whether or not they worked — a cancelled finding is a public wrong).

Slot	Harare Metro's answer
Core question	Which services work, for whom, at what cost — provably?
Unit of analysis	The service interaction (permit, collection, connection); the ward-month; the programme-beneficiary
Key metrics	Service level (collections/permits per target), unit cost, programme coverage and leakage, citizen wait times
Data reality	Legacy systems, paper registers, ward-level politics in every count, periodic national surveys
First project	The service-level baseline: one service, measured honestly, published
Failure mode	Indicators that become targets (Goodhart) and evaluation that becomes advocacy

57.2 The Service Baseline and the Trust Problem

The first project in any public analytics practice is establishing a *baseline anyone can check*: for one service — refuse collection is the classic — the fleet's GPS tracks, the ward collection schedule, and a complaint log, reconciled into a single published number: "86% of scheduled collections achieved, by ward, this quarter". The trust problem is the real opponent: public-sector data has been politicised before, and the analyst's craft is *showing the work* — methodology published, data extractable, numbers reconcilable by any journalist with a spreadsheet. Chapter 42's lineage discipline is not a nice-to-have here; it is the difference between a dashboard and a press release. **From Your Toolkit — Power BI**: the public dashboard is a Power BI (or open-source equivalent) product with one unusual requirement: every published figure must carry its provenance link, because the audience includes adversaries by design.

57.3 Programme Evaluation Culture

Chapter 61's methods belong here at full strength: difference-in-differences, synthetic control, and the interrupted time series are the public sector's workhorses, because randomisation is usually politically impossible. The Metro's discipline, learned the hard way:

1. **Pre-register the analysis** — the question, the comparison, and the success metric, dated before the data arrive. In politics, the temptation to redefine success afterwards is structural, and pre-registration is the structural answer.
2. **Respect the comparison's honesty** — the ward next door is not a control group until you show it trends in parallel (DiD's assumption, checked and displayed, per Chapter 61).
3. **Publish the nulls** — the street-lighting programme that did not move crime statistics is a finding the next budget needs; suppressing it converts an error into a tradition.
4. **Bound the estimate** — "reduced reported complaints by 15% $\pm$ 8, with reporting artefacts unquantified" is the honest register; point estimates without intervals are lobbying with extra steps.

57.4 Operational Analytics: Queues and Registers

The service layer is Chapter 66's queueing in civic clothing: the licensing office is a queue (the Wq arithmetic that turned "we feel busy" into staffing plans in Chapter 54 works verbatim here), and the register is the product — a permit tracked from application to issuance, with the stage-level timestamps that expose where the month of delay actually lives. The public-sector twist is *adversarial demand*: the queue includes people who will pay to skip it, so process analytics (where the bottlenecks are) must be paired with integrity analytics (who skips, how often, at which stage) — Section 57.5's toolkit applied inward.

57.5 Procurement Analytics

Public procurement is where the money leaks, and the anomaly discipline (Chapter 46) has its highest civic value here: contracts clustered just below the tender threshold, single-bidder patterns by supplier, price drift against market indices, and the "split purchase" signature (one requirement, many small awards). The craft is the same three-layer stack as fraud: rules that catch the blunt patterns (explainable, auditable, publishable), a model for the residue, and a *human investigation* layer — because in the public sector, a flagged case that survives review must be presentable to an audit committee, and explainability is the law of the genre. Chapter 53's decline-letter discipline transposes: every flag carries its reasons, in language a non-technical councillor can repeat.

57.6 Open Data and Privacy as Engineering

Publication is the public sector's default and its hardest engineering problem: aggregate enough to protect (Chapter 54's small-cell suppression), document enough to verify (methodology, extract dates, known gaps), and version enough to trust (yesterday's number must still be reconstructable when the auditor asks — Chapter 42 again). The rare-data floor: administrative data on citizens carries the same harm test as health data (Chapter 54), and Chapter 68's formal techniques are the far end of the same road. The practitioner's rule: **publish the aggregate, protect the row, and write down which is which.**

57.7 Failure Modes

- **Goodhart at national scale** — collections-per-ward targets met by redefining "scheduled"; every indicator needs its companion audit metric (Chapter 45's pairing, now with an audit committee in the room).
- **Evaluation as advocacy** — the programme evaluation written to survive a cabinet meeting rather than an audit; pre-registration and published nulls are the antidotes.
- **The dashboard nobody uses** — the beautiful expenditure dashboard the councillors never open, because it answers no question anyone holds; Chapter 19's audience discipline is civic, not corporate.
- **The extract that becomes the record** — a one-off data pull, exported and quoted, drifting from the live system; version and date everything.

Teaching Tip — The adversarial reading: have students publish (in class) a one-page service baseline, then assign a third of the room to attack it as the opposition councillor: What is hidden? What is redefined? What would you FOA-request? The defending team learns more about methodology documentation in twenty minutes of adversarial reading than in a week of lectures — and it is exactly the muscle the job requires.

Key Takeaways

- Public analytics has many masters and permanent scrutiny: show the work — lineage, methodology, extractable data — because the audience includes adversaries by design.
- The first project is one honest, published service baseline; trust is the deliverable as much as the number.
- Evaluation culture: pre-register, check the parallel-trends assumption, publish the nulls, and bound every estimate.
- Procurement anomaly analytics is civic work: explainable flags, auditable reasons, presentable cases.
- Publish the aggregate, protect the row, and write down which is which.

Practice Lab

1. Build the refuse-collection service baseline from the fleet GPS and ward schedules (Appendix F generator); publish the methodology page and the reconciliation table, then run the adversarial reading exercise on a classmate's baseline.
2. DiD practice: evaluate the street-lighting expansion on ward-level incident data; display the parallel-trends check honestly and write the finding — including the null if that is what the data says.
3. Queueing at the licensing office: given arrival and service logs, compute the stage-level delays and the Wq arithmetic for two staffing options; write the memo with the p90 wait, not the mean.
4. Procurement anomalies: run the threshold-clustering and split-purchase rules on three years of awards; produce three flags with written reasons, each presentable to a non-technical committee.
5. The open-data release: take a service register and produce the publishable aggregate — suppression applied, methodology written, version dated — plus the internal note on what was withheld and why.
6. The indicator redesign: pick one current civic target, describe its Goodhart failure mode, and propose the paired metric that closes it.

Further Reading

- *Public Sector Analytics* literature: the What Works centres (the UK's movement is the reference model)
- Chapter 61 (causal inference), Chapter 42 (lineage), Chapter 46 (anomaly detection), Chapter 68 (formal privacy)
- Appendix J Case 4 (the tariff evaluation) — this playbook's deepest worked case

Chapter 58: Manufacturing Analytics — The Sabvura Industries Playbook

Part X — Analytics Across Industries: Domain Playbooks

"The factory already knows; the data's job is to get the knowledge out before the shift ends."

In this chapter you will learn

- The manufacturing canvas: throughput, quality, and uptime as one OEE story.
- Overall Equipment Effectiveness: the three losses and the availability × performance × quality arithmetic.
- Statistical process control: control charts that separate signal from shop-floor noise.
- Predictive maintenance: the survival-model discipline for machines that age.
- From forecast to line: production scheduling under Chapter 47's optimisation.
- Failure modes: local OEE, the cherry-picked chart, and the sensor swamp.

58.1 The Manufacturing Question

Sabvura Industries is a fictional garment manufacturer: three production lines (cutting, sewing, finishing), 410 operators, an export contract with a European buyer whose audits arrive unannounced, and a factory manager who keeps a whiteboard she calls "the truth" — hourly counts, defects, and downtime, written by hand. The analytical questions: **are the lines producing what they could** (OEE), **is quality stable or drifting** (SPC), **which machines will fail and when** (maintenance), and **what should we make next week** (scheduling). The distinctive feature of manufacturing data: it is generated *on the line, in real time, by machines and operators who will tell you the truth if the system makes truth-telling easy* — and the analyst's first job is usually to make the whiteboard digital without killing its honesty.

Slot	Sabvura's answer
Core question	Throughput, quality, uptime — where do the hours go?
Unit of analysis	The machine-hour; the lot/batch; the operator-shift
Key metrics	OEE and its three components, defect rate by defect class, mean time between failures, schedule adherence
Data reality	Machine counters, operator log sheets, QC inspections by lot, maintenance records in prose
First project	The downtime Pareto, machine-logged and reconciled
Failure mode	Local optimisation: a line that hits OEE by pushing defects downstream

58.2 OEE: The Three Losses

Overall Equipment Effectiveness is the sector's headline metric because it multiplies the three ways a shift is lost:

- **Availability** — run time ÷ planned time. Losses: breakdowns, changeovers, waiting for material.
- **Performance** — actual output ÷ output at rated speed. Losses: micro-stops, slow cycles.
- **Quality** — good units ÷ total units. Losses: defects and rework.

The arithmetic is honest and humbling: a world-class 85% OEE is $0.90 \times 0.95 \times 0.995$ — three decent numbers multiplied into an excellence few reach; Sabvura's finishing line runs 58%, which is three mediocre numbers multiplied into a management conversation. The craft is decomposition: OEE as a single number is a scoreboard, OEE *split into its three components by line and shift* is a diagnosis, and the downtime Pareto is where the first improvement project always lives.

```
SELECT line, shift_date, shift,
       SUM(run_minutes) / SUM(planned_minutes)           AS availability,
       SUM(actual_units) / NULLIF(SUM(rated_units), 0)    AS performance,
       SUM(good_units) / NULLIF(SUM(actual_units), 0)     AS quality,
       SUM(run_minutes) / SUM(planned_minutes)
       * SUM(actual_units) / NULLIF(SUM(rated_units), 0)
       * SUM(good_units) / NULLIF(SUM(actual_units), 0) AS oee
FROM line_counters
GROUP BY line, shift_date, shift;
```

58.3 Statistical Process Control

SPC is Chapter 14's inference, industrialised: a control chart is a hypothesis test drawn repeatedly through time, separating common-cause noise (leave it alone; tampering adds variance) from special-cause signals (find the reason, now). The discipline that survives every software fashion:

```
def xbar_chart(measurements, subgroup=5, sigma_k=3):
    import numpy as np
    groups = [measurements[i:i+subgroup] for i in range(0, len(measurements), subgroup)]
    means = np.array([g.mean() for g in groups])
    cl, sigma = means.mean(), np.array([g.std(ddof=1) for g in groups]).mean()
    ucl, lcl = cl + sigma_k * sigma / np.sqrt(subgroup), cl - sigma_k * sigma /
    np.sqrt(subgroup)
    out = [g for g, m in zip(groups, means) if m > ucl or m < lcl]
    return cl, lcl, ucl, out # out-of-control subgroups, each with its hours
```

The rule that gives SPC its power: **respond to signals, leave noise alone**. The classic manufacturing sin is adjusting a process that was stable — adding variance while feeling busy. The analyst's contribution is annotating the chart (which signal coincided with the fabric-batch change, the operator changeover, the humidity spike) so the investigation starts warm.

58.4 Predictive Maintenance

Machines age, and the honest model of aging is Chapter 63's survival analysis: each machine's hazard of failure as a function of age, run-hours, and condition signals (vibration, temperature, error codes). The discipline matters more than the model: survival curves give *probabilities over time windows* ("30% failure hazard in the next 200 run-hours"), not the false-precision "fails Thursday"; maintenance scheduling is then Chapter 47's optimisation (schedule the intervention to minimise expected downtime cost, given the hazard and the parts lead time). The cautionary tale every plant lives: the model that flags a failure risk the technicians already schedule around — predictive maintenance's value is over and above *experienced maintenance planning*, and proving that requires the same holdout discipline as any model (Chapter 23's league table, greasy edition).

From Your Toolkit — Excel: the maintenance backlog and the MTBF league table live happily in Excel pivot tables, and that is the right first home — the maintenance planners already work there. The survival model informs the planners; it does not replace them, and the interface is a table they trust.

58.5 From Forecast to Line

The scheduling layer closes the loop with the demand side: Chapter 28's order forecasts become Chapter 47's constrained schedule — lines, changeovers (whose cost in minutes is the real currency), operator rosters, and the export buyer's audit windows. The craft is making constraints explicit rather than implied: the intern's optimisation from Appendix J Case 3 failed not in its arithmetic but in its *unstated* constraint (the loading rule); the same lesson walks this factory's floor. A schedule no one can explain is a schedule the floor will quietly override — explainability is operational, not cosmetic.

58.6 Failure Modes

- **Local OEE** — the cutting line hits its target by passing borderline fabric to sewing; OEE must be measured at the *system* level, or the target destroys the plant.
- **The cherry-picked chart** — the control chart shown only in its stable months; keep the full run and the annotations, always.
- **The sensor swamp** — instrumenting everything and analysing nothing: start from the decision (which changeover? which intervention?) and instrument backwards from it.
- **The whiteboard's revenge** — the new digital system that operators feed garbage because truth-telling became harder than the paper it replaced; watch the reconciliation gap, and fix the form before blaming the floor.

Teaching Tip — The paper OEE: run one class session entirely on a paper log sheet: students record a simulated line's hour (planned, run, units, defects, reasons for stops) by hand, then compute OEE and the Pareto themselves. The lesson is double: they internalise the arithmetic *and* they feel how data quality is won or lost at the point of entry — the whiteboard manager's problem, lived for an hour.

Key Takeaways

- OEE is a scoreboard; its three components by line and shift are the diagnosis, and the downtime Pareto is the first project.
- SPC is repeated inference: respond to special-cause signals, leave common-cause noise alone — tampering is the industrial original sin.
- Predictive maintenance is survival curves plus scheduling optimisation, and must beat experienced planners in a fair holdout to count.
- Constraints belong in the model, not in the lore; a schedule the floor cannot explain is a schedule it will override.
- Data quality is won at the point of entry: make truth-telling easier than the paper it replaced.

Practice Lab

1. Compute OEE and its components by line and shift from the line counters (Appendix F generator); build the downtime Pareto and name the first improvement project with its expected minutes.
2. Build the X-bar chart for seam strength by line; annotate the out-of-control subgroups against the batch and operator logs; write the two investigation memos the chart demands.
3. Fit the survival model for the cutting machines (age, run-hours, vibration); produce each machine's 200-hour hazard window, and the maintenance schedule that minimises expected downtime cost.
4. The league-table test: does your survival model beat the planners' existing PM calendar in a backtest? Design the fair comparison and report the result honestly.
5. The scheduling round: given the order book, changeover matrix, and rosters, produce next week's schedule — then write the one-page explanation the floor supervisor reads aloud at the shift briefing.
6. The reconciliation audit: compare the digital counters against the whiteboard's week; quantify the gap by reason code, and propose the form change that closes the largest one.

Further Reading

- *Out of the Crisis* — W. Edwards Deming (SPC and the tampering doctrine)
- *OEE for Operators* — Nakajima (the source text, short and complete)

- Chapter 47 (scheduling optimisation), Chapter 63 (survival analysis), Chapter 14 (the inference beneath SPC)

Chapter 59: Transport and Logistics — The Kunaka Logistics Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Logistics is the mathematics of promises: an ETA is a contract signed by a queue."

In this chapter you will learn

- The logistics canvas: routing, reliability, fleet health, and the cold chain.
- The constraint discipline of routing optimisation — and why the optimiser's world is not the world.
- ETA prediction: the regression that respects the road.
- Fleet analytics: utilisation, maintenance, and the cost per delivered case.
- IoT data engineering for cold-chain compliance: the temperature excursion ledger.
- Failure modes: average speed, the unmodelled constraint, and the dashboard that lies to the customer.

59.1 The Logistics Question

Kunaka Logistics is the fictional cold-chain operator you met in Appendix J Case 3: one depot, 62 retail drop points, 14 vehicles, and a dairy client whose product fails quality control if any drop arrives more than four hours after loading. Its analytical questions: **which stops on which vehicle in which order** (routing), **when will it actually arrive** (ETA), **what does each vehicle really cost** (fleet economics), and **did the cold chain hold** (compliance). The distinctive feature: logistics is the discipline of *constraints wearing costumes* — time windows, temperature ramps, loading order, driver hours — and the recurring failure of logistics analytics is building a beautiful model of a world without them.

Slot	Kunaka's answer
Core question	Which routes, in what order, at what reliability and cost?
Unit of analysis	The stop; the trip; the vehicle-day
Key metrics	Cost per delivered case, on-time %, distance per drop, temperature excursions, fleet utilisation
Data reality	GPS traces, telematics temperature logs, drive-time matrices, driver manifests on paper, traffic by hour
First project	The on-time baseline and the excursion ledger
Failure mode	Optimising distance in a world that runs on time

59.2 Routing: The Constraint Discipline

Vehicle routing is a canonical optimisation problem — and a canonical warning. Case 3's story is the sector in miniature: an 18% distance saving that became 11% once drive-times replaced distances and the four-hour constraint was enforced, then lost two routes entirely to the loading-order rule nobody wrote down. The practitioner's method:

1. **Write the constraints as a list, with owners.** Time windows, capacity, cold-chain ramps, driver hours, the customer's receiving dock that closes at 16:00. Each constraint gets a name and a source — the union rep's sentence about loading order is data.
2. **Solve with the simplest method you can explain.** Cluster-then-sequence (group stops by geography, then order each cluster greedily by time) is usually within a few percent of the optimum and can be explained to a driver — which is worth more than the missing percent.
3. **Validate by simulation, not assertion** (Chapter 67): Monte Carlo the plan under demand and traffic variance; read the p10 day. The plan that survives the p10 day is the plan; the rest is a diagram.
4. **Pilot on one route for two weeks** before signing the network. The pilot is where the unmodelled constraints introduce themselves.

59.3 ETA Prediction

The customer-facing layer is a regression problem with one unusual property: **its error has a sign that matters**. An ETA five minutes early is a promise kept; five minutes late is a promise broken, and the asymmetric cost belongs in the loss function — predict the *quantile* that makes lateness rare enough to match the service promise (Chapter 25's quantile thinking, delivered). The features are the road's honest structure: distance and traffic by hour, stop durations learned from GPS dwell times, the dock queue at the receiving end, and the trip's own history at that hour.

```
model = GradientBoostingRegressor(loss="quantile", alpha=0.85) # 85th percentile
model.fit(features, arrival_deltas_minutes) # positive = late
# The promise: "arrives by X" where X is the 85th-percentile arrival.
```

From Your Toolkit — Excel: the ETA *audit* is a pivot table the operations team can own: promised versus actual arrival by route and hour, the lateness rate, and the worst three stops — the analysis that keeps the model honest lives in the tool the dispatcher already has open.

59.4 Fleet Economics

The unit that disciplines everything is **cost per delivered case** — fuel, driver, maintenance, depreciation, and the cold-chain energy, divided by cases delivered — because it exposes the trade the fleet runs every day: an under-filled van at 40% capacity has a beautiful cost-per-kilometre and a terrible cost-per-case. The analytics: utilisation by day and route (the half-empty Tuesday run), maintenance cost per vehicle against age and odometer (Chapter 63's survival hazard, feeding the replace-or-repair decision), and the fixed-versus-variable split that answers the lease question. The quiet discipline: allocate costs to trips honestly (including the depot overheads), or the route-level profitability ranking is fiction with decimals.

59.5 The Cold-Chain Ledger

Cold-chain compliance is an IoT data engineering problem before it is an analytics one: temperature sensors stream intervals; excursions must be detected, attributed (which stop's door opening?), and reported to the dairy client within the contract's window. The artefact is the **excursion ledger** — every trip's temperature trace, every breach flagged with the stop, duration, and peak, and the reconciliation against the quality standard. The engineering is Chapter 9's pipeline discipline (validation gates on sensor data — stuck readings, gaps, clock drift) with Chapter 46's anomaly logic on top: the sensor that reports a perfectly flat 4.0°C for three hours is not a healthy freezer; it is a broken sensor, and the ledger must treat it as a data incident, not a compliance pass.

59.6 Failure Modes

- **Average speed** — routing on mean drive-times schedules the fleet for a world without the 16:00 truck; drive-time *distributions* by hour are the honest input.
- **The unmodelled constraint** — Case 3's lesson: constraints live in drivers' heads and unions' rules until they are written as data.
- **The dashboard that lies to the customer** — an on-time metric defined on departure rather than arrival; every customer-facing metric must name the clock it uses.
- **Sensor silence as health** — the flat line is an incident; validation gates make silence loud.

Teaching Tip — The drivers' meeting: have students present a route redesign to a panel playing drivers, dispatchers, and the union rep (Case 3's cast). The panel's first question is always a constraint the model missed — and the lesson, that the person closest to the work knows the model's missing variable, lands as lived experience rather than a maxim. Rotate roles; the next cohort's modellers are this cohort's drivers.

Key Takeaways

- Constraints are the logistics model: list them with owners, solve with methods you can explain, validate by simulation on the p10 day, and pilot before signing.
- ETA models predict service, not averages: fit the quantile that matches the promise, and audit promised-versus-actual in the dispatcher's own tool.
- Cost per delivered case disciplines the fleet; allocate overheads honestly or the route ranking is fiction.
- The cold-chain ledger treats sensor silence as an incident — validation gates first, compliance second.
- The person closest to the work knows the missing constraint; ask before optimising.

Practice Lab

1. Build the on-time baseline and excursion ledger from the telematics extract (Appendix F generator); report on-time % by route and the three worst stops, naming the clock each metric uses.
2. Refit the Case 3 routing with drive-time distributions and the loading constraint; report the honest saving and the two assumptions you most want validated in the pilot.
3. Fit the ETA quantile model; choose the service quantile, and show the lateness rate it implies against the customer's contract threshold.
4. The fleet economics table: cost per delivered case by vehicle and route; identify the under-filled run and the replace-or-repair candidate from the maintenance hazard curve.
5. The sensor audit: run the validation gates on a month of temperature logs; document the flat-line sensors, the clock drift, and what the compliance number becomes once they are excluded.
6. Run the drivers' meeting on your own redesign; log every constraint the panel adds, and revise the model and its explanation in one page.

Further Reading

- *Vehicle Routing* — Toth and Vigo (the reference; read Chapter 1 and the heuristics chapters)
- Appendix J Case 3 (the van in the wrong place) — this playbook's full worked case
- Chapter 47 (optimisation), Chapter 67 (simulation), Chapter 25 (quantiles for service promises)

Chapter 60: Media and Sport Analytics — The Nhaka FC and Kumba Playbook

Part X — Analytics Across Industries: Domain Playbooks

"Sport measures everything and wins nothing by measuring; media measures nothing and calls it reach. Both are wrong the same way."

In this chapter you will learn

- The two-headed canvas: performance analytics (sport) and attention analytics (media).
- Sport: expected goals in miniature, recruitment value, and injury risk as a survival problem.
- Media: the metrics that pay — attention, churn, and content decay.
- Kumba's frontier: analytics and language technology for low-resource markets.
- The storytelling bridge: how analytics becomes broadcast.
- Failure modes: Goodhart ruins the game, and reach is not revenue.

60.1 The Two-Headed Question

This playbook pairs two industries that share one underlying commodity — **attention** — and one shared failure: measuring what is easy instead of what matters. **Nhaka FC** is a fictional Premier Soccer League club: 28 professional players, an academy, modest revenue, and a technical director who wants to stop signing players "because the chairman saw them once". **Kumba** is a fictional media and language-technology company: streaming radio and news in Shona and Ndebele, a small advertising business, and a language-services division whose frontier work — Chapter 65's low-resource NLP — is watched from well beyond Harare. The canvas, twice:

Slot	Nhaka FC's answer	Kumba's answer
Core question	Which players, which style, which risks — at what wage?	Which content, for whom, at what attention value?
Unit of analysis	The match event; the player-season; the session	The listener-session; the content item; the campaign impression
Key metrics	Expected goals for/against, points per wage, availability rate	Listening hours, subscriber churn, attention per content dollar
Data reality	Event-coded matches, GPS training loads, sparse injury records	Streaming logs, app telemetry, no panel ratings, Shona/Ndebele content untitled
First project	The recruitment value ledger	The content decay curve
Failure mode	Metrics that ruin the playing style	Reach that never becomes revenue

60.2 Sport: Expected Goals in Miniature

The idea that changed football analytics: shots are not equal, so count the *quality* of chances. A minimal expected-goals (xG) model — the shot's outcome probability given location, body part, and assist type — is a logistic regression you can fit in an afternoon and defend in a pub:

```
import statsmodels.formula.api as smf
model = smf.logit("goal ~ distance + angle_rad + header + fast_break",
                  data=shots).fit()
shots["xg"] = model.predict(shots)
# A season's story: goals minus xG = finishing (or luck) above chance creation.
```

The honest uses: xG over a season separates *chance creation* from *finishing luck* (a striker 6 goals above xG is due a regression the wage negotiation should know about); xG against exposes a defence that has been bailed out by its goalkeeper; and the rolling xG difference is the earliest signal that the team's style has changed. The honest caveats: sample sizes are small (a season is ~350 shots), the model is only as good as the event coding, and — the classic — xG is a *scouting* tool, not a tactic: a team that optimises shot-taking to maximise xG becomes predictable (Section 60.6's Goodhart, in boots).

60.3 Recruitment and Injury

Recruitment value is the club's moneyball ledger: contribution (xG+xA per 90, availability-weighted) per wage dollar, compared honestly against the *replacement* — the academy player who costs a stipend. The craft is the comparison set: a winger scouted against the whole league looks average; against age-mates in the same tactical role, he is a bargain or a fraud. **Injury analytics** is Chapter 63's survival discipline in tracksuit: hazard of injury as a function of training load (GPS distance, high-intensity efforts), congestion (matches per 14 days), and history — used to *manage exposure* (rotate before the hazard spikes), never to promise that a player will stay fit. Availability is the quiet metric: the squad's expected available minutes is worth as many points as any signing.

60.4 Media: Attention That Pays

Kumba's analytics stand on one honest distinction: **reach is not attention, and attention is not revenue**. Reach counts devices; attention measures minutes; revenue needs *minutes that advertisers or subscribers will pay for*. The core artefacts:

- **The content decay curve:** listening by content item against minutes-since-publication, by format — news decays in hours, drama serials in days, and the shape (not the peak) tells the schedule what to commission next.
- **The churn layer:** Chapter 27's retention curves on subscriber data — which programmes appear in the listening histories of those who stay? (Correlation, honestly labelled; the causal test is Chapter 61's — stagger a schedule change across regions and watch.)

- **The attention ledger:** listening hours per content dollar, by format and language — the commissioning meeting's one table.

From Your Toolkit — Power BI: the attention dashboard is a genre you know: cohort retention curves, a decay chart per format, and the commissioning table on the front page. The bridge: this is the same retention analytics as Zuva Mobile (Chapter 52) with listening minutes in the ARPU seat.

60.5 Kumba's Frontier: Language Technology

Kumba's deepest analytics problem is Chapter 65's subject and this part's bridge to Part XI: content in Shona and Ndebele is poorly served by tools trained on the internet's English — transcription, search, and recommendation underperform exactly where Kumba's audience lives. The practical frontier: speech and text models adapted to low-resource languages (transfer learning from related languages, SentencePiece tokenisation on Shona corpora), metadata generated in-language (the untitled archive is unsearchable in any language), and — the quiet craft — *evaluation sets built by hand* because no benchmark exists. The strategic lesson for every analyst: where the tooling is thin, the data you can build becomes the moat.

60.6 The Storytelling Bridge and the Shared Failure

Analytics in both industries becomes valuable when it becomes *story*: the broadcast graphic (xG rolling difference at minute 70), the commentary line, the schedule insight a producer can pitch. Chapter 74's structures apply verbatim: one message, evidence, the decision. And both industries share the terminal failure mode — **Goodhart**: the player who stops shooting from anywhere because low-xG shots dent the dashboard; the newsroom that chases clicks into trivia. The practitioner's guard is the paired metric (Chapter 45): every performance metric travels with its health companion — xG with unpredictability, clicks with subscriber retention — and the pair is reviewed together, or the metric eventually ruins the thing it measured.

Teaching Tip — Code your own match: give students a raw match event file (20 minutes of an amateur match, coded from video) and have them compute xG, then explain it to a panel playing a sceptical coach, a chairman, and a journalist. The three explanations must differ — the coach wants tactics, the chairman wants money, the journalist wants a sentence — and the exercise teaches the last mile of this whole part: the same analysis, told three ways, or it was never done.

Key Takeaways

- Sport and media both measure attention; both fail by measuring the easy proxy.
- xG is a small, defensible model with big honest uses — chance creation versus finishing luck — and it is a scouting tool, not a tactic.
- Recruitment value is contribution per wage against the honest replacement; availability is the quiet signing.

- Reach is not attention, attention is not revenue: decay curves, churn curves, and the attention per content dollar ledger are the media desk.
- Where tooling is thin — low-resource languages — the data you build is the moat.
- Pair every metric with its health companion, or Goodhart ruins the game and the newsroom alike.

Practice Lab

1. Fit the minimal xG model on the shot data (Appendix F generator); report each striker's goals minus xG and write the wage-negotiation paragraph that regression-to-the-mean deserves.
2. Build the recruitment ledger: contribution per 90, availability-weighted, per wage dollar — against the whole league and against age-mates; note where the two comparisons disagree and which you trust.
3. Fit the injury hazard model on training loads and congestion; produce next month's rotation plan and its expected available minutes.
4. Build the content decay curves by format and language; write the commissioning memo — what to renew, what to retire, what to schedule at 18:30.
5. The churn layer: retention curves for subscribers with and without each flagship programme in their history; label the correlation honestly and design the staggered test that would settle it.
6. Code 20 minutes of a match (any match, your own events file) and produce the three explanations: coach, chairman, journalist.

Further Reading

- *The Numbers Game* — Anderson and Sally (football analytics' best entry point)
- *Soccernomics* — Kuper and Szymanski (recruitment economics)
- Chapter 63 (survival for injuries), Chapter 65 (low-resource NLP — Kumba's deep end), Chapter 74 (the three-way storytelling)

APPENDICES

Reference Section

Quick answers, cheat sheets, solutions and practice data

Appendix E: Datasets and Resources

Practice data, public datasets, and the reading list behind the course

"Data is the course's tuition: every dataset below is chosen because it teaches something specific."

How to Use This Appendix

Every chapter's Practice Lab assumes data. Three sources cover the whole course: **Appendix F's Soulfya's generator** (fabricates the running case's entire universe with planted patterns), the public datasets listed here, and your own organisation's extracts (used under Chapter 42's governance and Chapter 68's privacy floor). Work the labs in that order: generated data first (you know what should be found), public data second (you practise finding it), real data last (nothing teaches governance like real data).

When you download a public dataset, practise the Part II discipline from the first minute: write the data card (source, licence, grain, known gaps), load it through your validation gates, and only then begin analysis. The habit is the point — a dataset you have not profiled is a stranger you are about to marry.

Core Public Datasets, by Part

Dataset	Where	Use it for	Part
Soulfya's generated suite	Appendix F script	Every running case lab	I–XII
Titanic passenger list	Public (Kaggle, many mirrors)	First classification, missing data, leakage-safe practice	V
Ames / Boston housing	Public (scikit-learn, Kaggle)	Regression, feature engineering, honest evaluation	III, V
NYC taxi trips	NYC Open Data	Big-ish data, DuckDB/Spark practice, fares and geography	VII
Bikeshare trips (Capital Bikeshare or local)	Open data portals	Time series, seasonality, forecasting leagues	III, V
Online Retail II (UCI)	UCI ML Repository	Market basket, RFM, CLV — the Part X retail labs	IV, X
Telco customer churn	Public (IBM sample, Kaggle mirrors)	Churn baselines, calibration, uplift practice	V, X
Bank marketing (UCI)	UCI ML Repository	Uplift and campaign response, imbalanced targets	V, X
Olist Brazilian e-commerce	Kaggle	Multi-table joins, star schema, delivery ETAs	II, VII, X
Superstore sample	Tableau Public	Dashboard drills, metric trees	IV
Gapminder / World Bank indicators	gapminder.org, data.worldbank.org	Trend craft, small multiples, development analytics	III, IV, X

Dataset	Where	Use it for	Part
DHS / MICS survey data (aggregated)	dhsprogram.com, unicef.org/mics	Survey weights, stratified estimation	III, X
CRAN / UCI time series	UCI, Monash TS archive	Forecasting league tables	V
Medical cost / diabetes readmission (UCI)	UCI ML Repository	Clinical risk, calibration	X
Palmer penguins	Public (allisonhorst)	First EDA, tidy graphics	I, IV
Open FIFA / football events	Public (Kaggle)	Expected goals, sport analytics	X

Licence note: check each source's licence before publishing derived work; academic-teaching use is broadly permitted for most of the above, but "public" is not "yours to redistribute" — the data card records the licence with everything else.

Tools (Free), by Purpose

- **Python stack:** pandas, scikit-learn, statsmodels, matplotlib, seaborn, statsforecast or sktime for leagues; DuckDB for analytical SQL on files.
- **Big data practice:** DuckDB first (single-node is the honest default, Chapter 37); Spark locally via pyspark when the decision test says so; Dask rarely and deliberately.
- **Dashboards:** Power BI Desktop (free), or Plotly Dash / Streamlit (Chapter 49) for products.
- **Causal inference:** statsmodels, causalimpact, dowhy, econml (Part XI's frontier reading).
- **Survey analysis:** Stata or R with survey; Python's statsmodels with weights for the same discipline.
- **Geospatial:** geopandas, libpysal (Moran's I and friends, Chapter 62).
- **Survival:** lifelines (Chapter 63).
- **Experimentation:** scipy.stats, statsmodels sequential tests, and Chapter 67's simulation notebooks.
- **Text:** sentence-transformers, sentencepiece (Chapter 65's low-resource work).

Domain Playbook Addendum (Part X)

Each playbook chapter pairs with the datasets above and one domain-specific practice source. The pattern: run the chapter's Practice Lab twice — once on the Soulfya's-generated analogue (where Appendix F plants the answers), once on the public source (where the world decides).

- **Retail (Ch. 51)** — Online Retail II for basket and RFM; Olist for the multi-table version; the dead-stock ledger lab runs on Appendix F's inventory snapshot.
- **Marketing (Ch. 52)** — Telco churn and Bank marketing for the uplift and cohort labs; any app-store or SaaS sample for payback-week practice.
- **Financial services (Ch. 53)** — The UCI credit-approval style samples and Give Me Some Credit (Kaggle) for scorecard practice; generate fraud with Appendix F's

planted-adversary mode.

- **Healthcare (Ch. 54)** — Diabetes 130-US readmission (UCI) is the canonical readmission lab; any DHS/MICS extract for the survey-weight discipline.
- **Agriculture (Ch. 55)** — FAOSTAT for production series; CHIRPS rainfall (public) and MODIS NDVI for the layered forecast; Sable Fields comes from Appendix F.
- **Energy (Ch. 56)** — UMass/ISO-NE or ENTSO-E public load data for the day-ahead forecast league; smart-meter samples (UK-DALE, Pecan Street) for interval analytics.
- **Government (Ch. 57)** — Your own city's open-data portal (Harare's and most capitals publish something); World Bank indicators for the evaluation labs; Case 4's tariff data is generated.
- **Manufacturing (Ch. 58)** — The SECOM dataset (UCI) for SPC-style quality; Appendix F's line counters for OEE and the maintenance hazard.
- **Transport (Ch. 59)** — NYC taxi for routing-adjacent ETA practice; any GTFS feed for schedule reality; Kunaka's telematics is generated (Case 3's numbers).
- **Media and sport (Ch. 60)** — Public football event data (Kaggle's European matches) for xG; any podcast/streaming sample for decay curves; Kumba's corpora are built, not downloaded (Chapter 65).

Frontier and Practitioner Resources (Parts XI-XII)

- **Causal inference (Ch. 61)** — Scott Cunningham's *Causal Inference: The Mixtape* (free online); the World Bank's DiD resources; *causalimpact* documentation.
- **Geospatial (Ch. 62)** — *geopandas* gallery; the Geographic Data Science textbook (free); *libpysal*'s spatial-statistics tutorials.
- **Survival and forecasting depth (Ch. 63-64)** — *lifelines* documentation (an excellent tutorial in itself); FPP3 (*Forecasting: Principles and Practice*, free online) for hierarchical methods; *sktime* for the leagues.
- **Low-resource NLP (Ch. 65)** — Masakhane's published work (the reference for African-language NLP); *sentencepiece* papers; Hugging Face's course.
- **Simulation (Ch. 66-67)** — Julia/Python queueing tutorials; the Monte Carlo kitchen notebooks from Chapter 67; any discrete-event library's manual (SimPy) is a course in itself.
- **Experimentation (Ch. 67)** — Kohavi, Tang and Xu, *Trustworthy Online Controlled Experiments* — the practitioner's bible; CUPED references from Microsoft's published papers.
- **Privacy (Ch. 68)** — The Differential Privacy textbook (Dwork and Roth, free online); OpenDP's documentation.
- **Analytics engineering (Ch. 69)** — *dbt*'s documentation and blog; Kimball's dimensional toolkit.
- **Staying current (Ch. 70)** — The newsletters and reading discipline of Chapter 70; Papers With Code for method tracking.
- **Delivery craft (Ch. 71-80)** — Appendix J's five cases as the role-play corpus; *The Flaw of Averages* (Savage) for the constraint of the week; the options-paper and

value-ledger templates from Part XII, used on your own engagements.

Building Your Own Practice Data

When no public dataset fits (often, for Parts VII-XII), generate it: the discipline is Appendix F's — write down every planted pattern before generating, so the analysis can be *scored against truth*, which no real dataset offers. A generated dataset with documented plants is a better teacher than a real dataset with unknown structure; the sequence is always generated → public → real, in that order, for every new skill.

Appendix I: Instructor Slide Kit

Twelve starter decks — one per part — for teaching the course

"Slides are promises about time: each one says 'this many minutes, this one idea'."

How to Use This Kit

Each deck below is a skeleton: slide titles, the one message per slide, and the teaching note that anchors it. Build them in PowerPoint, Google Slides, or (for the reproducible) Marp or Quarto — the structure survives any medium. The decks are deliberately short: 8–12 slides each, one idea per slide, so that the lecture's remaining minutes belong to the live demo and the lab, where learning actually happens.

General deck rules (Chapter 17 applies to slides too):

- One message per slide, stated as the slide's title — a claim, not a label ("Joins duplicate fact rows" beats "Joins").
- Evidence on the slide: one chart or one table, never both.
- Speak the takeaway; the slide carries only what words cannot — numbers, shapes, structure.
- End every deck with the lab brief: what students do in the next ninety minutes.

Deck 1 — Part I: Foundations of Big Data Analytics

1. **Title.** Course name, your name, the promise: from six tools to one discipline.
2. **Analytics is decisions, not data.** The question ladder: decision → question → data → analysis → decision again.
3. **The maturity ladder.** Descriptive → diagnostic → predictive → prescriptive; where the value and the difficulty both live.
4. **When is data big?** The Vs, honestly: most "big data" problems are big-mess problems.
5. **Data types and where they hide.** Structured, semi-structured, unstructured; JSON in the wild.
6. **The six-tool bridge.** SQL, Excel, Python, Power BI, Stata, SPSS — one grammar, six dialects.
7. **Soulfya's Restaurant Group.** Meet the running case: three outlets, one loyalty programme, a delivery channel, and every pattern in this course.
8. **Lab brief.** The data inventory.

Teaching note: the whole deck is one idea — questions before data. Everything else in Part I is vocabulary for that idea.

Deck 2 — Part II: Data Engineering for Analysts

1. **Title.** Part II — where trustworthy data comes from.
2. **Grain first.** The single question that prevents most analysis errors: what does one row mean?
3. **Joins.** Inner/left/anti — and the fan-out that multiplies revenue.
4. **Star schemas.** Facts and dimensions; why analysts build wide tables from narrow ones.
5. **Pipelines.** Extract, validate, load; the pipeline that runs is worth ten that are elegant.
6. **The pandas-to-SQL bridge.** Same grammar, different engine; push work to the data.
7. **Cleaning is contract negotiation.** Names, nulls, duplicates, units — each one a business question.
8. **Quality gates.** Validation as code; trust is a pipeline property.
9. **Lab brief.** Build Soulfya's star schema.

Teaching note: run the duplicated-orders demo live — the gasp when revenue triples is the whole part.

Deck 3 — Part III: Mathematics and Statistics

1. **Title.** Part III — how to argue with numbers.
2. **Descriptives lie honestly.** Mean/median/spread; Anscombe's lesson in four datasets.
3. **Probability by counting.** The joint table; Bayes as arithmetic on a grid.
4. **The logic of a test.** Hypothesis, statistic, p-value, decision — and what a p-value is not.
5. **Simulation over formula.** The coin-flip lab: watch false positives appear.
6. **Sampling.** Random, stratified, weighted; the exit poll that went wrong.
7. **Regression.** One line, three stories; slope as rate, residuals as information.
8. **Lab brief.** The Bayes table; the permutation test.

Teaching note: never teach the formula first. Count first, simulate second, formula third — the formula then arrives as a shortcut, not a mystery.

Deck 4 — Part IV: Visualization and BI

1. **Title.** Part IV — charts that change decisions.
2. **Form follows question.** The chart-choice matrix: comparison, distribution, composition, relationship, trend.
3. **Pre-attentive craft.** Colour, position, length; ink spent on the message.
4. **The metric tree.** Goals → drivers → metrics; dashboards as forests, not trees.
5. **Dashboard genres.** Operational, tactical, strategic — one audience, one question, one screen.

6. **Power BI as product.** The toolkit bridge: relationships, measures, bookmarks.
7. **Storytelling.** Context → conflict → resolution; the executive summary that survives the stranger test.
8. **Lab brief.** The one-page dashboard.

Teaching note: bring bad charts (there are repositories of them) and let students perform the audit — critique teaches faster than admiration.

Deck 5 — Part V: Machine Learning Foundations

1. **Title.** Part V — the modelling loop.
2. **Framing.** Classification, regression, ranking, clustering — which question is this?
3. **Data before models.** Split logic, target leakage — the leak that flatters and lies.
4. **Baselines.** Majority class, seasonal naive, last year: the cheapest honest competitor.
5. **Linear models.** The line, regularized; interpretability as a feature.
6. **Evaluation.** The metric matches the decision: precision/recall trade-offs, calibration, cost curves.
7. **Trees and ensembles.** Splits, forests, boosting; variance and bias made visible.
8. **Ethics from day one.** Fairness metrics, disparate impact, the model card.
9. **Lab brief.** The leaky-feature audit; the baseline league table.

Teaching note: the planted-leak dataset (Appendix F) is the part's centrepiece — let students discover the AUC of 0.99 and then take it apart.

Deck 6 — Part VI: Advanced Machine Learning

1. **Title.** Part VI — beyond the first model.
2. **Clustering.** Segmentation as hypothesis generation, not truth.
3. **Recommenders.** Collaborative filtering; the cold start and the popularity trap.
4. **NLP.** Text as features: bag-of-words to embeddings.
5. **Deep learning.** When depth earns its keep; data, compute, and the humility curve.
6. **LLMs.** Tokens, prompts, fine-tuning; the analyst's assistant, not the analyst.
7. **Explanation.** SHAP, permutation importance — explaining models to the people who live with them.
8. **Lab brief.** The segmentation and recommender double lab.

Teaching note: keep the deep-learning segment honest with the "when NOT to" slide — the discipline is the lesson.

Deck 7 — Part VII: Big Data Technologies and the Cloud

1. **Title.** Part VII — what actually changes at scale.

2. **The decision test.** Size, speed, structure, spend — most problems are single-node problems.
3. **DuckDB.** The analytical database on a laptop; push the work down.
4. **Spark.** Partitions, the shuffle, lazy execution — distributed thinking.
5. **Streaming.** Windows, watermarks; the dashboard that is always now.
6. **Cloud economics.** Storage, compute, egress; the bill as design feedback.
7. **Lab brief.** The Spark job on Soulfya's delivery events.

Teaching note: open with the same dataset in pandas and DuckDB and let the stopwatch make the argument for the part.

Deck 8 — Part VIII: From Insight to Impact

1. **Title.** Part VIII — running analytics responsibly, forever.
2. **Governance.** Stewardship, lineage, privacy; trust as infrastructure.
3. **MLOps.** The pipeline, the registry, the smoke test; reproducibility as a habit.
4. **Monitoring.** Drift, decay, the champion-challenger rhythm.
5. **Ethics under load.** Consent, harm, the refusal clause.
6. **Careers.** The craft ladder: analyst → senior → lead → principal; the T-shaped professional.
7. **Lab brief.** The model card and the monitoring plan.

Teaching note: this deck lands best after the capstone has broken something — governance reads differently mid-incident.

Deck 9 — Part IX: The Extended Curriculum

1. **Title.** Part IX — special topics and the craft of teaching.
2. **Anomaly detection.** Rules, distances, isolation; the two error costs of fraud.
3. **Optimization.** From prediction to prescription; the roster that writes itself.
4. **Graph analytics.** Entities, edges, communities; influence beyond the average.
5. **Data products.** The rung ladder: report → dashboard → app → API.
6. **Teaching.** Backwards design; the lecture-lab-feedback rhythm.
7. **Lab brief.** Ship one data product.

Teaching note: this part is where students become colleagues — teach it as a seminar, not a lecture.

Deck 10 — Part X: Analytics Across Industries — Domain Playbooks

1. **Title.** Part X — ten industries, one grammar.
2. **The playbook canvas.** Core question, unit of analysis, key metrics, data reality, first project, failure mode — six slots, every industry.
3. **Retail and marketing.** Basket, churn, CLV, uplift; the promotion that pays for itself.

4. **Financial services and health.** Credit risk, fraud, readmission; the regulated industries, where explanation is law.
5. **Agriculture and energy.** Yield, weather, load, tariffs; forecasting where the weather is a feature.
6. **Transport, manufacturing, government, media.** Routing, OEE, programme evaluation, attention; each playbook in six slots.
7. **The pattern across playbooks.** Same grammar, different data realities — the analyst's real skill.
8. **Lab brief.** Fill the canvas for a local organisation.

Teaching note: run this part as case-method seminars — Appendix J's five cases are the syllabus; the deck only frames the method.

Deck 11 — Part XI: The Frontier — Advanced Methods and Emerging Practice

1. **Title.** Part XI — where the working core ends.
2. **Causal inference.** DiD, IV, synthetic control; the question correlation cannot answer.
3. **Space and time.** Moran's I, survival curves, hierarchical forecasts; the structures between rows.
4. **Language and simulation.** Low-resource NLP, synthetic data, agent-based markets.
5. **Experimentation at scale.** Bandits, CUPED, sequential tests; the infrastructure of learning.
6. **Privacy engineering.** Differential privacy; sharing data safely.
7. **Staying current.** The reading discipline; ten predictions to track.
8. **Lab brief.** One frontier replication.

Teaching note: one chapter, one paper, one replication — depth over survey; the frontier changes, the discipline of entering it does not.

Deck 12 — Part XII: The Practitioner's Path — Projects, Clients and the Craft of Delivery

1. **Title.** Part XII — analytics as a delivered service.
2. **The project arc.** Scope, plan, analyse, deliver; the stranger test at every phase.
3. **Engagements.** The options paper; pricing and the refusal clause.
4. **Stakeholders.** The question ladder; the no-surprises rule.
5. **Storytelling and documentation.** Three structures; the four artefacts; the analysis log.
6. **Audits, constraint, onboarding.** Ten audit questions; the \$200 stack; the first ninety days; the trust ledger.
7. **The business of the craft.** The value ledger; productization; compounding practice.

8. **Lab brief.** The full engagement role-play.

Teaching note: the role-play is the assessment — hostile questions, scope creep, a pricing decision, and a refusal, all in one session.

Deck Hygiene Checklist

Before any deck ships:

- ☐ One message per slide, stated in the title
- ☐ Every chart follows Part IV's craft rules
- ☐ Code screenshots never exceed half a slide
- ☐ The lab brief is the final slide
- ☐ You have spoken the deck aloud once, with a timer

Twelve decks, one course, roughly 110 slides — about two semesters of teaching material when each deck anchors a lecture and the labs carry the rest. Adapt freely: the skeletons are yours. The course, like the book, ends where your own practice begins.

Appendix J: Extended Case Portfolio

Five end-to-end cases for seminar use, capstone inspiration, and interview preparation

"Cases are compressed experience: you live a project's hardest week in ninety minutes."

How to Use These Cases

Each case is written to be attacked, not admired. The recommended seminar format is the case method: students read the case cold (15 minutes), identify the decision and the decision-maker (10 minutes), work the analysis in small groups (30 minutes), then defend their recommendations against challenge (35 minutes). The teaching notes at the end of each case list the traps — the places where the obvious answer is the wrong one.

For interview preparation, use them differently: pick a case, work it alone under a timer, write the one-page memo, and then deliver the executive summary aloud. Chapter 74's structures and Chapter 73's question ladder apply to cases exactly as they apply to live engagements.

The five cases share a world — they can be run in sequence as a single course-long narrative, or independently. All data referenced is generated by Appendix F's Soulfya's generator or by the small generator scripts included in each case; all numbers reconcile across cases, so a cohort can build a shared "institutional memory" of the fictional companies.

Case 1: The Promotion That Ate the Margin

Soulfya's Restaurant Group — the Soul Circle double-points promotion

Domain: uplift modelling, causal reasoning, experiment design — Chapters 24, 32, 61, 62

The Situation

Azu, the founder of Soulfya's, is staring at a report she commissioned and half-regrets. Six months ago, the marketing team ran a double-points promotion through the Soul Circle loyalty programme: members who visited during the promotion window earned twice the usual points. The post-mortem report shows a triumph — visits among promoted members rose 31% versus the prior quarter, and the campaign was hailed internally as the best in the programme's history.

But Azu has two doubts. First, the finance director points out that the promotion's cost (points redeemed, discount processing, campaign administration) rose faster than revenue, and gross margin among promoted members fell 4.2 percentage points. Second, Azu has just read Chapter 61 of this book, and one line is stuck in her head: *the counterfactual is the customer who would have come anyway*.

The loyalty programme has 48,000 members. During the six-week promotion window, 11,200 members visited at least once and received the promotion. The remaining 36,800 members were not targeted — the campaign was sent by email, and only members with an email address on file were eligible. A junior analyst's quick check shows that email-holding members skew higher-income and more urban.

You have member-level data: visit logs for 52 weeks before and 6 weeks after the promotion, redemptions, satisfaction surveys, and a channel flag (email on file or not). You are hired to answer one question: **did the promotion create value, and should it run again?**

The Work

Your group must produce:

1. **A re-analysis of the 31% lift** — decomposed into what the promotion caused and what would have happened anyway. Email-eligibility selection bias must be addressed; a simple before-after comparison is not acceptable.
2. **An uplift segmentation** — following Chapter 32's four-cell logic: persuadables, sure things, lost causes, sleeping dogs. Estimate the proportion of the 11,200 in each cell (methods: matched controls from the non-email pool, or a model trained on the treated with matched untreated comparators; defend your choice).
3. **A margin-aware value calculation** — the promotion's true incremental profit, using the finance director's numbers: average margin among promoted members fell 4.2 points; points redemption cost averaged \$1.85 per redeeming member; campaign administration was \$9,000.
4. **A recommendation memo** — one page to Azu: run again, kill, or redesign; if redesign, the targeting rule (which cells to target), the control discipline (how the next run is measured), and the expected value of the redesign.

The Numbers (for the instructor)

The generator plants: a true treatment effect of +9% visits among persuadables (about 58% of the treated), zero effect among sure things (about 22%), and a small negative effect among sleeping dogs (about 8%, who deferred visits to earn points they then let lapse). The remaining 12% are lost causes. The 31% headline lift decomposes as: +9 percentage points true uplift among persuadables, +18 points of selection bias (email members were visiting more anyway and the pre-quarter was seasonally weak), +4 points of regression to the mean. The honest incremental profit is negative \$23,400 — the promotion lost money — but a redesigned campaign targeting only persuadables (identified by pre-period visit recency and frequency) is worth about \$31,000 per run with a 10% holdout for measurement.

Teaching Notes

The traps: (1) the before-after comparison, which every group tries first — let them, then ask what the non-email members did that quarter; (2) averaging uplift across cells — the sleeping dogs' deferral is invisible in the average; (3) forgetting the margin — revenue lift with margin collapse is a well-dressed loss; (4) recommending "run it again but bigger" — the case is designed so that bigger means worse. The strongest groups will ask for the holdout *before* being told it exists, and will design the next campaign as an experiment, not a broadcast.

Case 2: The Churn Model That Was Too Good

Soulfya's — the Soul Circle attrition model, second edition

Domain: leakage, evaluation honesty, model deployment — Chapters 22, 23, 25, 43

The Situation

Six months ago (see Chapter 22's framing), the first churn model went live: gradient boosting on behavioural features, AUC 0.83, used weekly by the retention team to target win-back offers. It has been a quiet success — churn among contacted members is down 1.8 points.

Now the data science team has built version 2. It is dramatically better: AUC 0.97 on the holdout. The team lead wants to swap it into production immediately; the retention manager is thrilled; Azu asks the question that gets you hired: "The model is nearly perfect. What is it seeing that we are not?"

You have: the v2 feature list (47 features), the training data dictionary, the holdout evaluation, and SHAP summary charts showing that three features dominate: `days_since_last_redemption`, `loyalty_status_current`, and `support_tickets_last_30d`.

The Work

1. **Audit the feature list for leakage** — for each of the 47 features, state the moment in time it becomes available, and whether that moment is before or after the prediction point (the model scores members on Monday to plan the week's outreach). The data dictionary shows `loyalty_status_current` is refreshed nightly, and a member's status becomes "lapsed" 30 days after their last visit — the same event that defines the churn target.
2. **Diagnose the AUC** — explain in plain language why the leak inflates the metric, and estimate the honest AUC by rebuilding the model with the three leaked features removed (provided: the team's rerun, AUC 0.79 — slightly *below* v1).
3. **The decision** — v2 is honestly worse than v1. Write the memo: what ships, what is communicated to the retention team, and what process change prevents v3 from repeating the error (Chapter 43's review gates).
4. **The harder question** — `support_tickets_last_30d` is *not* leaked (tickets precede the prediction point), but it is dominant. The retention manager asks: "Should we stop reading tickets and let the model do it?" Advise on the organisational risk of a model that quietly becomes the only thing anyone reads (Chapter 45's automation complacency).

Teaching Notes

Traps: (1) groups may accept the "honest" rerun as the fix and miss that v2-without-leaks is *worse than v1* — the decision is not "which model" but "why did we spend three months making a worse model", and the answer is evaluation without baselines; (2) the `days_since_last_redemption` feature is borderline — it is available at prediction time but correlates with the target's definition; a strong group distinguishes *timing* leaks from *definitional* leaks; (3) the automation-compliance question has no clean answer — reward groups that surface it rather than resolve it. The case's quiet lesson: a model that is too good is not lucky, it is lying.

Case 3: The van in the Wrong Place

Kunaka Logistics — cold-chain network redesign

Domain: optimisation, constraint analysis, simulation — Chapters 47, 67, 71

The Situation

Kunaka Logistics runs a 14-vehicle cold-chain distribution network for a dairy processor: one depot, 62 retail drop points, two vehicle classes (5 refrigerated vans, 9 ambient trucks that can carry insulated boxes), and a hard constraint — every drop must arrive within 4 hours of depot departure or the product fails quality control.

The operations manager has a spreadsheet optimisation (built by an intern) that recommends a new route plan projected to cut total distance by 18%. The board has approved the budget contingent on your independent review. The plan looks clean: routes are contiguous, distances check out, vehicle assignments respect capacity. But the drivers' union rep, invited to the review meeting "for completeness", says one sentence that stays with you: "Your plan has us loading the dairy before the pharmaceuticals."

The load sequence matters: pharmaceuticals must be loaded first (strictly controlled temperature ramp), and the plan's most efficient route has the pharma drop scheduled fourth — meaning 90 minutes of door openings before the pharma delivery, which the quality team says risks the temperature excursion limit.

You have: the 62 drop points with coordinates and demand, the current route plan, the intern's optimisation, drive-time matrices (with traffic by hour), the quality standard, and the union rep's account of loading practice.

The Work

1. **Reproduce the 18%** — verify the optimisation's arithmetic, then test its assumptions: the optimiser minimises distance, not time-under-constraint; rebuild with drive-time and the 4-hour constraint. How much of the 18% survives? (Provided generator: the honest saving is 11%, and two of the optimiser's routes violate the 4-hour window in afternoon traffic.)
2. **Model the loading constraint** — add the pharma-first rule; solve again (a simple heuristic — cluster then sequence — is acceptable and preferred to a black-box solver you cannot explain).

3. **Simulate before you sign** — Monte Carlo the recommended plan: demand $\pm 25\%$, drive times from the traffic model, one vehicle breakdown per week. What is the plan's service-level distribution? What does the p10 day look like? (Chapter 67's kitchen-sink method.)
4. **The recommendation** — one page to the board: what to approve, what to renegotiate (the intern's plan is not wrong, it is *incomplete*), and what to measure in the first month of operation.

Teaching Notes

Traps: (1) accepting distance as the objective — the case is engineered so that time-under-constraint reverses two route assignments; (2) the union rep is not an obstruction, he is the constraint's voice — groups that marginalise him miss the case's central lesson (Chapter 73: the person closest to the work knows the model's missing variable); (3) simulation is not optional — the deterministic plan's service level looks fine until the p10 day; (4) the intern's work should be *built on*, not discarded — Chapter 71's options-paper discipline: credit, correct, extend. The board does not need a new optimisation; it needs an honest one.

Case 4: The Anti-Model

Harare Metro Water — social tariff design

Domain: causal inference, ethics, policy — Chapters 45, 61, 63

The Situation

Harare Metro Water serves 1.2 million connections. A cross-subsidy tariff design is on the table: raise the volumetric rate on the top consumption decile to fund a lifeline tariff (first 5 kilolitres free) for the bottom two deciles. The engineering consultancy's model — a sophisticated microsimulation using 2019 consumption data — projects the policy is revenue-neutral and lifts 40,000 households above the water-poverty line.

The city council's data office (your client) has doubts about three things: the 2019 data predates two major tariff shocks that reshaped consumption; the model assumes no behavioural response to price changes (no elasticity); and the lifeline eligibility is based on registered household size, which audit samples suggest is under-reported by about 30% in exactly the households the policy targets.

Your brief is narrow and uncomfortable: **you have two weeks to say whether the model's projection can be trusted, not what the policy should be.**

The Work

1. **The data audit** — recompute the consumption distribution on post-shock data (provided: a 2023 sample of 40,000 connections). How much does the top decile's share move? (Planted: the top decile's consumption share falls from 34% to 29% — the subsidy base is smaller than projected, and the policy is no longer revenue-neutral; it runs a 6% deficit.)

2. **The elasticity bound** — the model assumes zero behavioural response. Use the two tariff shocks as natural experiments (difference-in-differences against a comparable metro's published data, provided) to estimate a price elasticity range for the top decile; run the policy under your range. (Planted: elasticity around -0.35; under it the deficit widens to 9-11%, because the subsidised decile conserves when priced up.)
3. **The eligibility problem** — model the consequence of 30% under-reporting: how much of the lifeline benefit leaks to households above the poverty line? What is the cost of verification (provided: a doorstep verification pilot cost of \$2.10 per household, and a 1.2 million-connection scale)? Is the policy still progressive after verification costs?
4. **The memo** — you were not asked for a recommendation, but Chapter 74's honest-practitioner rule says the analysis implies one. State what the model gets right, what it gets wrong, the bounded range of outcomes, and the decision the council actually faces (fund the deficit, verify eligibility, or redesign). Resist the temptation to decide the politics; do not resist the temptation to be useful.

Teaching Notes

Traps: (1) groups overstep the brief and become policy advocates — the teaching point is the boundary of the analyst's mandate (Chapter 71's scoping discipline), and the memo's final line should acknowledge it; (2) the elasticity estimate is genuinely uncertain — reward honest bounds over false precision, and expect a DiD table that students struggle to make clean (real natural experiments are messy; that is the lesson); (3) the under-reporting finding cuts *against* the policy's intended beneficiaries, which is uncomfortable — the case tests whether students report unfashionable findings; (4) "two weeks" is real: the deliverable is a bounded, honest memo, not a perfect study. The strongest close: "The model is trustworthy as arithmetic, and misleading as a projection; here is the projection range under defensible assumptions."

Case 5: The Grant That Would Not Close

Sable Fields Co-operative — programme evaluation under data loss

Domain: data engineering, missing data, survival analysis, integrity — Chapters 11, 15, 63, 75

The Situation

Sable Fields is a 2,400-farmer co-operative that took a four-year productivity grant. The funder's close-out report requires an impact evaluation: did grantee farmers' yields and incomes rise more than non-grantees'? The co-operative's manager delivers your data drop with an apology: "The tablets broke in year two. We have complete records for 2021, then nothing until mid-2023, then complete records again."

The dataset: 2,400 farmers (1,100 grantees, 1,300 non-grantees), annual yield and income records, plot sizes, and a participation log. The gap: year 2 (2022) is 78% missing, and — you discover in your first hour — the missingness is not random:

farmers with small plots were less likely to be re-enrolled in the record system after the tablet failure, and grantees with small plots were *more* likely to remain recorded (extension officers chased them for compliance photos). The co-operative's board needs the evaluation to succeed for the next grant; the funder's evaluator will pounce on any gap in the data story.

The Work

1. **Quantify the missingness** — build the missingness table: recorded vs unrecorded by grant status, plot size, and baseline yield. State the mechanism in Rubin's terms (Chapter 11's typology) and defend the classification: this is MAR conditioned on observables, not MNAR — the case is engineered so that the small-plot differential is fully explained by grant status and baseline yield.
2. **Choose and defend a strategy** — multiple imputation (chained equations) versus inverse-probability weighting versus complete-case with bounds. Run the evaluation under your primary strategy and at least one bound; Chapter 15's discipline applies: the sensitivity analysis is the result.
3. **The survival question** — the co-operative also lost farmers: 380 exited the co-operative during the grant period, and exit is correlated with poor yields (you observe exit dates for all 2,400). Naive comparisons of "remaining farmers" overstate impact. Model time-to-exit as a survival curve (Chapter 63), stratified by grant status, and reconcile: does the grant *reduce* exit, or delay it? (Planted: it delays it — the survival curves converge by year 4; the grant's honest effect on yields is +6.1% with CI [2.3, 9.8], and the effect on exit is a delay, not a prevention.)
4. **The close-out memo** — to two audiences at once: the funder's evaluator (technically literate, adversarial) and the co-operative's board (practical, anxious). One document, an executive summary that serves both, and an appendix that pre-empts the evaluator's ten questions (Chapter 75's audit list). Your integrity test: the memo must state the data gap in its first paragraph, not its last.

Teaching Notes

Traps: (1) complete-case analysis — which the case's construction makes deliver a +14% effect (the planted answer is +6.1%); the naive answer is more than twice the truth, and the board will love it; (2) imputation without the missingness table — the mechanism must be established before the strategy is chosen, or the imputation is decoration; (3) the survival analysis is optional-looking but load-bearing — the exit-delay finding changes the grant's story from "transformation" to "bridging", which is a defensible and fundable finding; (4) the two-audience memo is the hardest deliverable in the case portfolio — most groups write either a technical report or a pep talk, not both in one document. The integrity arc: the data loss is not the co-operative's shame, it is the evaluation's context — and honesty about it is what makes the +6.1% credible enough to fund the next grant.

The Portfolio's Arc

The five cases are one curriculum. Case 1 teaches that correlation dressed as a campaign report is still correlation. Case 2 teaches that models can lie with perfect accuracy. Case 3 teaches that optimisation without constraints is a plan for a world that does not exist. Case 4 teaches the boundaries of the analyst's mandate. Case 5 teaches that honesty about data's limits is the strongest thing an analyst can publish. Run in sequence, they are a capstone: a student who can work all five — the causal decomposition, the leakage audit, the constrained optimisation with simulation, the bounded policy memo, and the two-audience close-out with missing data — is ready for practice, and has the portfolio to prove it.

This was Appendix J — the final chapter/appendix of an eighty-chapter, twelve-part book. The course is complete: the grammar of the craft, the domains where it lives, the frontier that extends it, and the path of delivering it. What remains is practice — and the practice never ends, which is the good news. Go build the trustworthy pipeline, ship the honest model, write the memo that respects its reader, and teach someone else to do the same. The data is already pouring in somewhere near you.

— prepared for Tatenda Makuvaza, "*The Big Data Analyst*", 2026

About this Edition

Production notes

Big Data Analytics and Machine Learning — The Complete Practitioner's Course: From SQL, Excel and SPSS to Spark, Deep Learning and Production AI was typeset directly from its Markdown manuscript with `build_pdf.py` (ReportLab): 7 x 10 inch textbook trim, running headers, a linked table of contents, and PDF outline bookmarks.

- Edition: First Edition, 2026 (compiled 05 October 2026)
- Contents: 12 chapters in 12 parts, plus 3 appendices
- Volume: approximately 25,493 words of prose and 152 lines of code (estimated 101 print pages)
- Toolchain: Markdown manuscript, `build_book.py` (HTML edition), `build_pdf.py` (this PDF)
- Typeface: DejaVu (serif body, sans display, mono code)

To rebuild either edition after editing the manuscript, run `python3 build_book.py` then `python3 build_pdf.py`. The companion HTML edition is screen-optimised and regenerates in seconds.

Thank you for reading this book — and for teaching from it. The data is already pouring in somewhere near you.